
FSI AIxData Challenge 2024

공모전 기획서

팀명: 흑석동 물주먹 (방소연, 김규린, 김은선, 조영범)

작성일: 2024. 09. 29.

[목차]

1. 아이디어 기획 핵심내용(요약).....	3
2. 아이디어 구상 및 제안 배경	4
3. 활용 데이터 및 생성AI 모델 활용 방안	6
4. 아이디어의 독창성	9
5. 아이디어의 실현가능성.....	14
6. 아이디어의 문제 해결 가능성	16
7. 아이디어의 발전 가능성.....	23
8. 아이디어의 적용 대상 및 효과.....	25
9. POC 설계 개념도 및 주요 코드 설명.....	27

1. 아이디어 기획 핵심내용(요약)

아이디어명: 사용자 맞춤형 AI 금융 지식 답변 서비스

1.1 기존 문제점 및 배경 및 타겟층

현재 다양한 조사를 통해 대한민국 다수의 국민들이 금융 용어와 금융 상품에 대한 이해도가 낮다는 사실이 발표되었다. 특히, 청년 및 청소년층에서 이러한 금융 지식 부족은 심각한 문제로 대두되고 있다.

때문에 본 아이디어는 금융 지식(금융 이해력)이 낮은 사용자들을 대상으로, 해당 사용자들의 금융 지식 수준에 맞춰 맞춤형 답변을 생성함으로써, 금융 지식에 대한 이해도를 향상시킬 것을 기대한다.

1.2 아이디어의 차별점

사용자의 질문을 바탕으로, 질문 수준을 초급자(Newcomer), 초중급자(Learner), 중급자(Investigator)가 할 법한 질문 중 한가지로 분류한다. 누적된 질문 수준을 바탕으로 사용자의 금융 지식 수준을 실시간으로 판단한다. 이후 사용자의 질문에 대해 사용자의 금융 지식 수준 맞춤형으로 답변한다. 또한 여러 금융 지식 및 금융 상품에 대한 다양한 질문을 생성형AI를 활용하여 유연하게 답변할 수 있다.

1.3 생성AI의 역할

생성AI인 LLM으로 사용자의 금융 지식 수준 파악 및 질문에 해당되는 답변을 생성한다. 이때 RAG를 통해 LLM의 단점인 환각을 최대한 완화하며 답변의 신뢰성을 높이하고자한다. RAG 및 생성AI를 통해 1차적으로 답변이 생성되면, 이후 사용자의 금융 지식 수준에 맞게 해당 답변을 가공한다. 이를 통해 초급자에게는 더 쉽게 설명을 하도록 반영하고, 중급자에게는 상대적으로 어려운 금융 용어도 반영되게끔 맞춤형으로 답변한다. 또한 LLM이 기존 대화 이력을 바탕으로, 사용자의 관심 있는 부분을 파악하여 추후 이를 반영한 답변을 생성한다. 이를 통해 사용자의 지식 수준에 맞는 개인화된 답변을 제공하며 이 과정에서 축적된 데이터를 분석하여 서비스의 품질을 지속적으로 개선할 수 있다.

1.4 기대 방향

본 아이디어는 금융 지식 수준이 낮은 사용자의 금융 용어에 대한 두려움을 해소하고, 다양한 금융 상품에 대한 이해를 돕는다. 또한 다양한 금융 상품에 대하여 자유롭게 질문할 수 있는 매개체로서, 사용자의 금융 지식 수준 맞춤형 답변을 제공해 올바른 금융 상품에 대한 의사결정을 내릴 수 있도록 돕는다. 더 나아가 금융 시장의 활성화와 잘못된 금융 이해로 인한 금융 사고를 예방한다.

1.5 아이디어 구조도

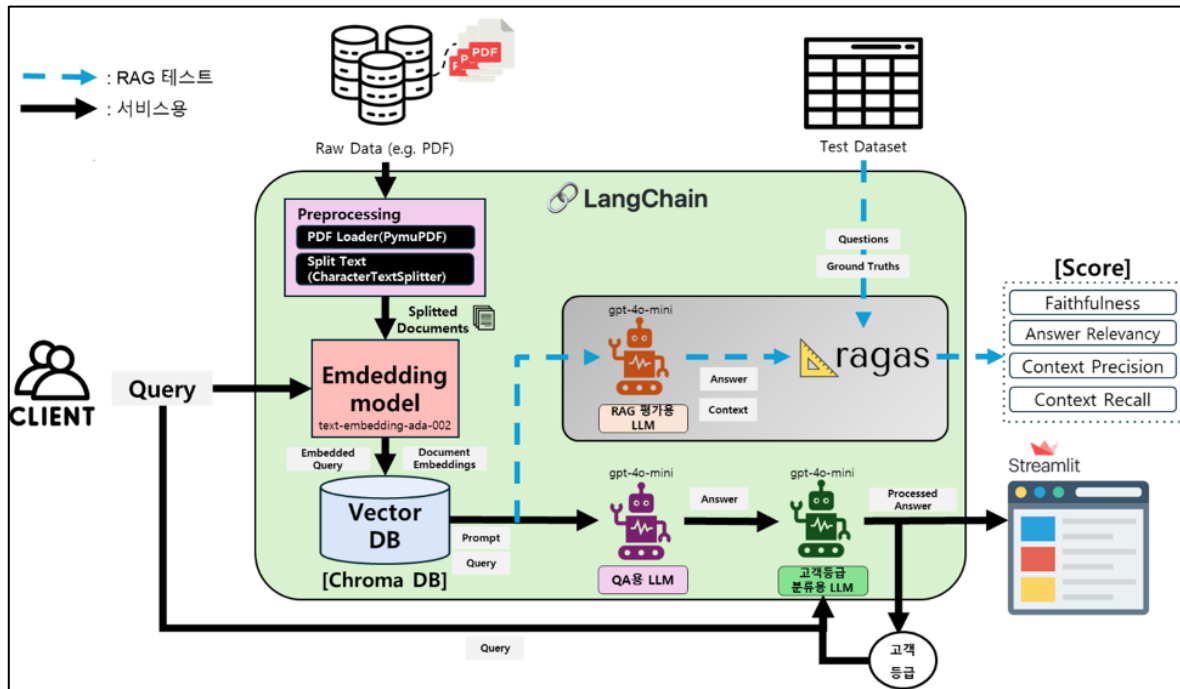


그림 1. 아이디어 PoC 구조도

2. 아이디어 구상 및 제안 배경

2.1 현재 고려되는 문제 배경

기획재정부가 발표한 ‘2023년 전 국민 경제 이해력 조사’¹에 따르면 대한민국의 경제 이해력 평균은 약 58점대로 60점에 미치지 못하는 수준이다. 또한 교육 수준 및 소득수준별 격차를 보이고 있다.

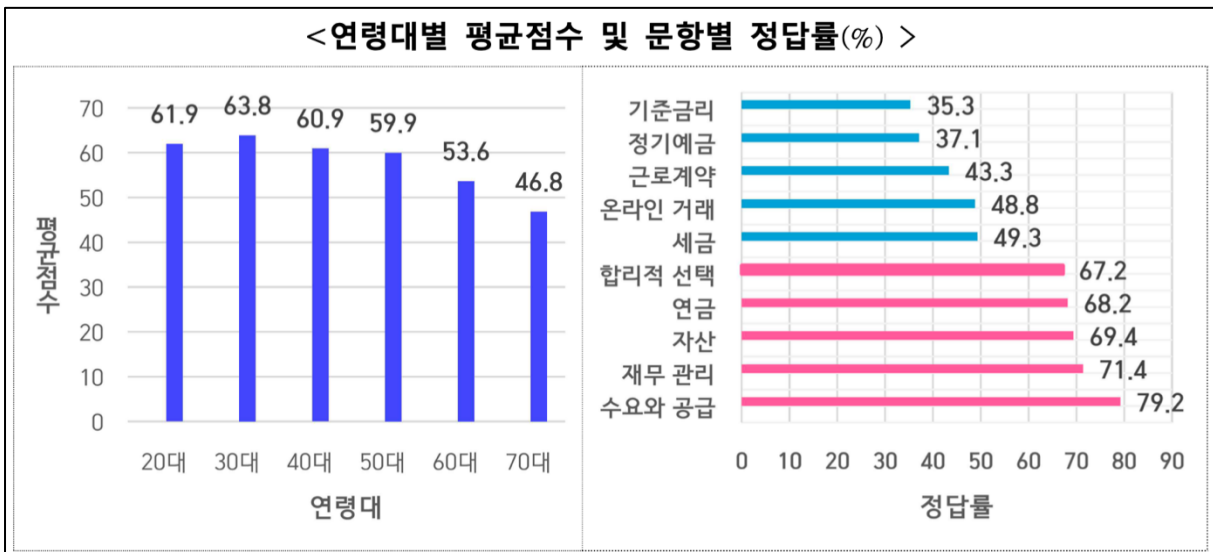


그림 2. 연령대별 경제 이해력 평균점수 및 문항별 정답률 그림

¹ <https://eiec.kdi.re.kr/policy/materialView.do?num=248622>

또한 최근 금융 이해력 관련 조사에서 청소년 및 청년층의 금융 지식 부족이 심각한 문제로 대두되고 있다. 2023년 조사²에 따르면, 한국 고등학생의 금융 이해력 평균 점수는 46.8점으로, 미국 금융교육기관 'Jump\$tart'가 설정한 기준인 60점에 크게 미치지 못했다. 이는 금융 이해력이 10년 전보다 더 악화된 수치로, 정규 교육 과정 내에서 금융 교육이 충분히 이루어지지 않음을 보여준다. 또한, 정부가 청소년을 대상으로 금융 교육 프로그램을 제공하고 있음에도 불구하고 실질적인 교육 효과는 미미하며, 금융 지식의 부족은 청년층의 경제적 의사결정 능력과 은퇴 준비에까지 영향을 미치고 있다.³

이러한 금융 교육의 미흡함은 청년층이 실생활에서 금융 상품이나 투자에 쉽게 접근하지 못하는 주된 요인 중 하나로 작용하고 있다. KB국민은행 조사⁴에 따르면, 10대들은 '돈 벌기'와 관련된 키워드를 자주 검색하지만, 실제로 금융 상품에 접근하는 데는 어려움을 느끼고 있는 것으로 나타났다. 이는 금융 지식의 부족과 더불어 전문 용어에 대한 이해 부족이 큰 진입 장벽으로 작용하고 있음을 시사한다. 실제로 보험 용어 관련 서비스를 포함한 금융 용어 해설 서비스들이 출시된 바 있지만, 사용자가 쉽게 접근하지 못해 활용도가 낮은 문제도 존재다.

현재 금융권은 이러한 문제를 인식하고 금융 이해도가 낮은 사람들 중 특히, 청년층을 대상으로 하는 맞춤형 금융 서비스를 제공하기 위한 다양한 노력을 기울이고 있다. 국민, 신한, 우리, 하나은행을 비롯한 주요 시중은행은 청소년 및 청년층을 겨냥한 금융 플랫폼을 개발 중이며, 세대별 눈높이에 맞춘 금융 언어 사용과 서비스를 적극적으로 추진하고 있다. 특히, 잘파세대(Z세대와 알파세대)를 포함한 청년층이 금융 시장에 진입할 수 있도록 용어를 단순화하고 맞춤형 금융 교육을 제공하는 것이 중요해졌다.⁵

2.2 생성AI를 통한 사용자의 금융 지식 수준 맞춤형 답변 생성

위와 같은 배경에서, 금융 지식 수준이 낮은 사용자들을 대상으로 한, '사용자 맞춤형 AI 금융 지식 답변 서비스'를 제안한다. 이 서비스는 사용자의 질문을 통해 사용자의 금융 이해 수준을 파악한 후, 이해하기 쉬운 용어와 개념을 기반으로 맞춤형 설명을 제공한다.

이 과정에서 언어 이해 및 추론 능력이 우수한 생성AI인 LLM을 활용한다. 사용자의 질

² '10년전보다 점수 더 떨어졌다...청소년 금융이해력 형편없다는데' - 매일경제 2023.05.31

³ 초고령 사회 임박..."청소년 금융-경제 교육 필수과목화해야" - 여성경제신문 2024.09.02

⁴ KB국민은행, 10만개 금융키워드 검색...10대 청소년, 돈 관심 급증 - MoneyS 2024.07.15

⁵ '미래 금융세대를 잡아라' 은행권 청소년 플랫폼 차별점은? - 토크데일리 2024.06.04

문을 이해한 뒤, LLM의 Few-shot learning을 통해 기존 정의된 질문 수준 유형을 판단한다. 이후 누적된 질문 수준 이력을 바탕으로 사용자의 금융 지식 수준 이해도를 실시간으로 반영한다. 사용자의 금융 용어 및 금융 상품에 대한 질문을 RAG 및 LLM을 통해 1차적으로 답변한다. 이후 측정된 사용자의 금융 지식 수준을 바탕으로 1차 답변에서 맞춤형 답변으로 가공하여 사용자에게 전달한다.

사용자에게 맞춤형 설명을 제공함으로써, 복잡한 금융 용어와 개념에 대한 장벽을 허물고, 금융 용어 및 금융 상품에 대한 접근성을 높인다. 이를 통해, 금융에 대한 막연한 두려움을 극복하고 금융 시장에 보다 자신 있게 진입할 수 있는 환경을 마련할 수 있다.

3. 활용 데이터 및 생성AI 모델 활용 방안

본 아이디어에는 예금, 적금, 보험, ETF 등 사용자가 관심 있는 여러 금융 상품들을 활용하여 사용자의 질문에 답할 수 있다.

하지만 PoC단계에서의 아이디어 구현을 위해, 본 기획서에는 여러 금융 상품들 중 사용자가 금융 ETF에 관심을 가지고 이에 관해 질문하는 것을 가정한다.

금융 ETF는 다른 금융 상품들에 비해 상대적으로 새로운 상품들이 빠르게 출시되고 있다. 많은 금융 상품의 효과적인 질의응답과 시장 수요에 맞추기 위해, TIGER ETF⁶의 간이 투자설명서와 월간운용보고서 각각 10개의 ETF 상품에 대한 PDF 파일을 샘플 데이터로 잡는다.

3.1 활용 데이터

최근 사용자들이 많이 찾는 ETF를 찾고자, 2024년 9월 6일 기준, 최근 1개월 수익률이 가장 좋은 각 테마별 2가지씩의 상품을 선정한다. 자세한 테마별 상품명은 아래 표 1과 같다.

표 1. 테마별 상품명

순번	테마	상품명
1	월분배	TIGER 미국30년국채프리미엄액티브(H)
2		TIGER 미국 MSCI리츠 (합성 H)
3	2차전지	TIGER 2차전지TOP10레버리지
4		TIGER 2차전지TOP10
5	전기차	TIGER 테슬라채권혼합Fn

⁶ <https://www.tigeretf.com/ko/theme-etf/list.do?>

6		TIGER 한중전기차(합성)
7	사이버보안/ 클라우드	TIGER 글로벌사이버보안 INDXX
8		TIGER 글로벌클라우드컴퓨팅INDXX
9	AI/반도체	TIGER 글로벌AI엑티브
10		TIGER 미국필라델피아반도체나스닥
11	메타버스	TIGER 글로벌메타버스엑티브
12		TIGER 게임TOP10
13	바이오	TIGER 바이오TOP10
14		TIGER S&P 글로벌헬스케어(합성)
15	클린 에너지	TIGER KRX기후변화솔루션
16		TIGER 탄소효율그린뉴딜
17	테크	TIGER 미국테크TOP10INDXX(H)
18		TIGER 미국나스닥100TR(H)
19	배당주	TIGER 배당성장
20		TIGER MKF배당귀족

간이투자설명서에는 투자목적, 투자전략, 주요 투자위험 등 해당 상품의 투자에 대한 간략한 설명이 있고, 월간운용성과에는 분배현황, 최근 운용성과, 성과 그래프 및 업종비중 등이 있다.

해당 설명서들을 바탕으로, 사용자가 묻는 상품에 대한 질문을 사용자 수준 맞춤형으로 답변하는 것을 목표로 한다. 또한 Large Language Model이 RAG를 통해 정확한 답변을 찾았는지 파악하기 위해, 평가 데이터셋을 구축한다. 사용자가 묻는 주요 질문들을 바탕으로 질문 목록 5가지는 아래 표2와 같다.

표 2. 평가 데이터셋 질문

순번	질문
1	해당 상품의 투자등급은 몇 등급입니까?
2	해당 상품의 기초 지수는 어떻게 되나요?
3	해당 상품의 투자 포인트는 무엇인가요?
4	해당 상품의 주요 종목 현황은 어떻게 되나요?
5	해당 상품의 운용 성과는 어떻게 되나요?

테마	상품명	url	답변1 (투자등급)	답변2 (기초지수)	답변3 (투자 포인트)	답변4 (주요보유현)	답변5 (운용성과)
월분배	TIGER 미국30년국채프리미엄액티브(H)	https	4등급 (보통 위험)	KEDI US Treasur	1. 금리 인하를 기대 2. 자본차익과 월배	iShares 20+ Year	최근 1달 3.33%, 최근 3달 8.64%, 최근 6달 3.49%
월분배	TIGER 미국 MSCI리츠 (합성 H)	https	2등급 (높은 위험)	MSCI US REIT In	1. 상장 리츠의 성장 2. 미국 상업용 부동산	Prologis Inc (10.1	최근 1달 5.03%, 최근 3달 18.27%, 최근 6달 13.41%
2차전지	TIGER 2차전지TOP10레버리지	https	1등급 (매우 높은 위험)	KRX 2차전지 TOF	1. 글로벌 주요 정부 2. 전기차 판매량 증	LG에너지솔루션 (3	최근 1달 4.59%, 최근 3달 -7.79%, 최근 6달 -41.38%
2차전지	TIGER 2차전지TOP10	https	1등급 (매우 높은 위험)	KRX 2차전지 TOF	1. 전기차, ESS 및 2. 글로벌 경쟁력을	LG에너지솔루션 (3	최근 1달 3.42%, 최근 3달 -1.57%, 최근 6달 -19.42%
전기차	TIGER 테슬라채권혼합Fn	https	4등급 (보통 위험)	FnGuide 테슬라채	1. 자율주행, AI, 반 2. 퇴직연금 내 100	Tesla Inc (29.50%	최근 1달 -4.16%, 최근 3달 6.79%, 최근 6달 4.72%
전기차	TIGER 한중전기차(합성)	https	2등급 (높은 위험)	KRX CSI 한-중 전	1. 한국 메모리와 중 2. 한중 우량 대표기	BYD Co Ltd (11.5	최근 1달 -3.00%, 최근 3달 -11.72%, 최근 6달 -10.34%
사이버보안/클라우드	TIGER 글로벌사이버보안 INDXX	https	2등급 (높은 위험)	Indxx Cybersecur	1. 디지털 전환 국면 2. 정부와 기업의 사	Global X Cybersec	최근 1달 1.65%, 최근 3달 3.56%, 최근 6달 2.20%
사이버보안/클라우드	TIGER 글로벌클라우드컴퓨팅INDXX	https	1등급 (매우 높은 위험)	Indxx Global Clo	1. 미래 IT 산업의 기 2. 늘어나는 채택근	Wix.com Ltd (4.3	최근 1달 -3.45%, 최근 3달 -2.06%, 최근 6달 -7.76%
AI/반도체	TIGER 글로벌AI액티브	https	2등급 (높은 위험)	Indxx Artificial Int	1. AI 산업의 강력한 2. 글로벌 AI 리딩 기	NVIDIA Corp (10	최근 1달 -0.83%, 최근 3달 -5.35%, 최근 6달 10.33%
AI/반도체	TIGER 미국필라델피아반도체나스닥	https	2등급 (높은 위험)	PHLX Semicondi	1. 4차산업혁명의 기 2. 비메모리 반도체	NVIDIA Corp (13	최근 1달 -4.60%, 최근 3달 -6.57%, 최근 6달 9.65%
메타버스	TIGER 글로벌메타버스액티브	https	2등급 (높은 위험)	Indxx Global Met	1. 모든 사회,경제,인 2. 메타버스를 선도	NVIDIA Corp (6.7	최근 1달 -0.90%, 최근 3달 -2.96%, 최근 6달 2.76%
메타버스	TIGER 게임TOP10	https	1등급 (매우 높은 위험)	KRX 게임 TOP 10	1. 콘텐츠 해외 수출 2. 글로벌 경쟁력을	크래프톤 (32.52%	최근 1달 2.27%, 최근 3달 5.57%, 최근 6달 7.60%
바이오	TIGER 바이오TOP10	https	1등급 (매우 높은 위험)	KRX 바이오 TOP	1. 글로벌 바이오 사 2. 글로벌 경쟁력을	SK바이오팜 (25.5	최근 1달 10.50%, 최근 3달 33.34%, 최근 6달 22.25%
바이오	TIGER S&P 글로벌헬스케어(합성)	https	3등급 (다소 높은 위험)	S&P Global 1200	1. 고령화와 건강 지 2. 헬스케어를 선도	Eli Lilly & Co (9.3	최근 1달 1.35%, 최근 3달 8.41%, 최근 6달 10.57%
클린 에너지	TIGER KRX기후변화솔루션	https	2등급 (높은 위험)	KRX 기후변화 솔	1. 저탄소 경제 전환 2. 저탄소 전환 및 기	삼성전자 (7.95%)	최근 1달 -0.30%, 최근 3달 -6.98%, 최근 6달 -7.28%
클린 에너지	TIGER 탄소효율그린뉴딜	https	2등급 (높은 위험)	KRX/S&P 탄소효	1. 국내 친환경 기 2. 환경에 대한 관심	삼성전자 (29.16%	최근 1달 -3.61%, 최근 3달 0.74%, 최근 6달 1.79%

그림 3. 평가 데이터셋: 질문과 ground truth 목록

3.2 활용한 생성AI 모델

사용자의 묻는 질문에 따른 적절한 판단 및 답변을 위해, 추론 및 판단 능력이 있는 생성AI모델인 Large Language Model(LLM) 3가지를 활용한다. 아래 3가지 모델 모두 Gpt-4o-mini 모델을 활용한다.

1) RAG 평가용 LLM: 사용자가 특정 상품에 대한 질문을 할 때, 모델이 RAG를 통해 적절히 검색하여 생성하는 모델이다. 실제 서비스에는 적용되지 않고, 모델의 성능향상 테스트 및 검증을 위해 활용된다.

2) QA용 LLM: 사용자의 묻는 질문에 대한 맞춤형 답변을 생성하는 모델이다. 상품에 대한 자료가 담긴 Vector DB에서 사용자가 묻는 질문에 대한 적절한 답변을 찾고, 사용자의 등급 분류용 LLM에 전달한다.

3) 사용자 등급 분류용 LLM: 사용자가 묻는 질문을 바탕으로 질문 수준을 판단한다. 또한 누적된 질문 수준을 바탕으로 사용자의 금융 지식 수준을 판단하여, QA용 LLM에서 생성된 답변을 맞춤형 답변으로 가공하여 전달하는 모델이다. Prompt에 입력한 Few-shot 예시들을 바탕으로 In-Context Learning을 통해 Newcomer, Learner, Investigator 3가지 중 1가지로 판단한다.

4. 아이디어의 독창성

본 제안서는 사용자 개개인의 금융 문해력 특성을 반영하기 위해, 맞춤형 금융 용어 및 상품에 대한 사용자의 질문을 답변한다.

4.1 제안하는 아이디어와 유사한 서비스

4.1.1 금융 용어 설명

1) 금융용어사전: 금융소비자 정보 포털⁷

- 538개의 금융용어에 대한 자세한 설명을 보여준다.

The screenshot shows the 'Financial Literacy' website. The search bar contains the text '찾고자 하는 용어의 일부 단어나 한글 자음 또는 알파벳(대문자)을 입력해도 검색이 가능합니다.' Below the search bar is a grid of letters (A-Z) and Korean consonants. The results section shows '538. 휴면예금 [Dormant Deposit]' and '537. 휴면보험금 [Dormant insurance]' with detailed descriptions of each term.

그림 4. 금융소비자 정보 포털의 금융용어사전 화면 이미지

2) 알기쉬운 금융용어 보기 서비스: 생명보험협회⁸

- 보험 상품 내의 어려운 금융 용어에 대한 설명(원어와 뜻, 예시 등)이 제시된다.
- 자연어 처리 AI 기술을 적용하여 정확한 단어 검출을 보여준다.

위 2가지 서비스 모두 사용자에게 금융 용어에 대한 설명을 보여준다. 그러나, 사용자의 금융 지식 수준을 고려하지 않고 동일한 설명을 제시한다. 또한 찾아볼 수 있는 금융 용어가 한정적이다. 즉, 현재 시장의 금융 용어 설명 서비스들은 사용자에게 일반화된 금융

⁷ <https://fine.fss.or.kr/fine/fncTip/fncDicary/list.do?menuNo=900021>

⁸ <http://news.bizwatch.co.kr/article/finance/2024/07/10/0016>

용어 설명을 제공하여 개인의 지식 수준 차이를 고려하지 못하고 제한된 범위의 정보만을 다루는 한계를 보이고 있다.

4.1.2 사용자의 금융 관련 수준 판별

1) 금융감독원 e-금융교육센터⁹

- 20문항을 통해 금융 지식 수준을 판단한다.
- 판단된 결과를 바탕으로 추천되는 금융 교육프로그램을 이수할 수 있다.

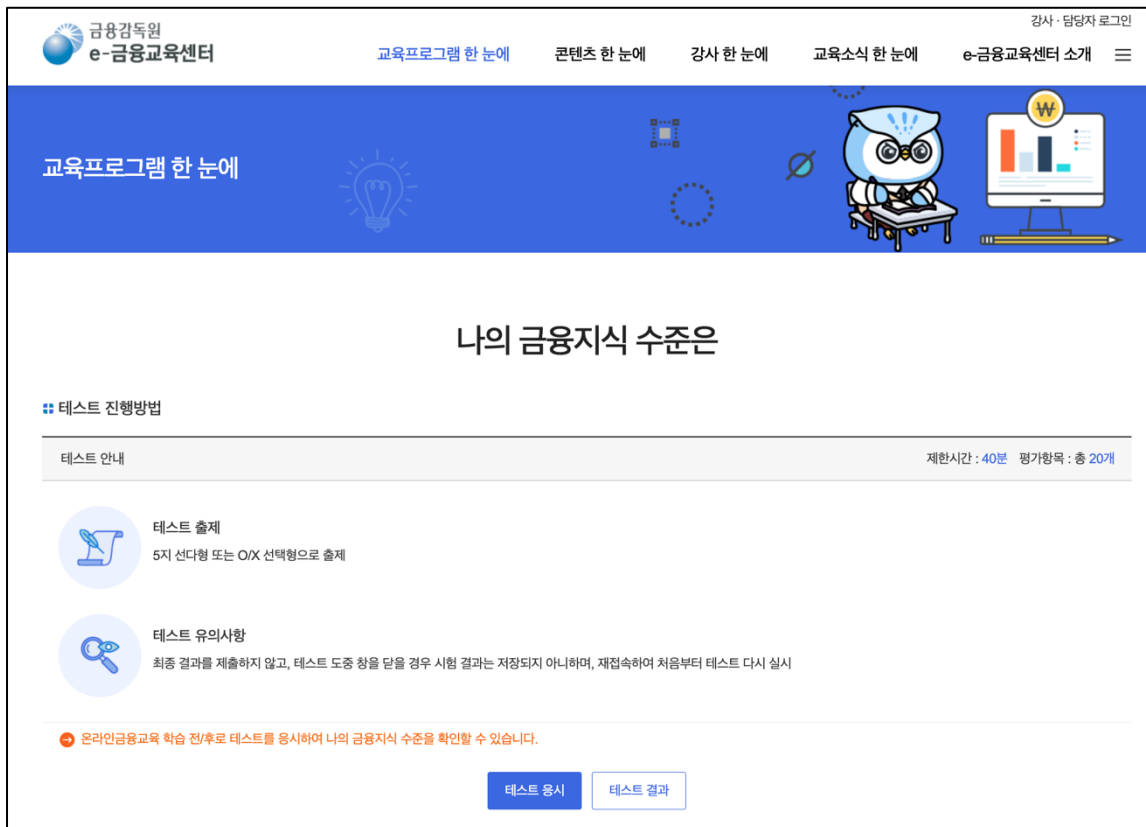


그림 5. 금융감독원 e-금융교육센터 금융 지식 수준 판단 화면 이미지

2) 사용자 투자 성향 분석

- 사용자가 적합한 투자 상품 가입을 위해, 사전에 투자 성향 자가 진단을 실행한다.

⁹ <https://www.fss.or.kr/edu/main/main.do?menuNo=300000>

위 2가지 서비스는 각각 사용자의 금융 지식 수준 및 투자 성향을 판단하는데 활용된다. 위 테스트 및 자가진단을 통해 사용자 스스로 금융 수준에 대한 이해도를 파악할 수 있지만, 이를 바탕으로 사용자가 궁금해하는 질문에 대한 답변을 하지 못한다는 단점이 있다.

4.1.3 개인 맞춤형 AI금융상품 추천 서비스

여러 금융 기관에서 개인 맞춤형 관련 서비스를 출시하고 있다. 최근 농협은행에서는 개인맞춤형 'AI금융상품추천서비스'¹⁰를 출시하였다. 해당 서비스는 딥러닝을 활용하여 고객에게 적합한 금융상품을 추천한다.

본 아이디어와 '개인 맞춤형'이라는 공통적인 특징이 있지만, 해당 서비스는 금융상품 추천에 초점을 맞추어 운영되고 있다. 이에 비해, 본 기획서의 아이디어는 사용자의 금융 이해력 증진을 목표로 제안되고, 기본적인 금융용어 뿐만 아니라 특정 금융 상품에 대한 폭넓은 설명을 각기 다른 금융 수준을 고려한 맞춤형으로 답변한다.

4.2 본 기획서의 차별화된 장점

본 기획서인 사용자 맞춤형 AI 금융 지식 답변 서비스는 위와 달리, 사용자의 금융 지식 수준을 실시간으로 분석하여, 맞춤형 학습 경험을 제공한다. 이러한 기능은 다음과 같은 다섯 가지 핵심 요소를 통해 차별화된 가치를 제공한다.

1) Newcomer, Learner, Investigator로의 사용자 유형 구분

본 기획서는 사용자의 질문 내용을 바탕으로 금융 지식 수준에 따라 세 가지 단계로 분류한다. 기존 금융 서비스들이 주로 투자 경험이나 보유 자산을 기준으로 사용자를 구분하거나, 미리 정의된 진단 테스트로 했던 것과 달리, 본 서비스는 사용자가 사용하는 금융 용어의 복잡성, 질문의 구체성과 깊이를 종합적으로 고려해 순수하게 금융 지식 수준과 이해도만을 기준으로 Newcomer, Learner, Investigator의 세 단계로 사용자를 세분화한다.

Newcomer는 금융 분야가 완전히 생소한 사용자로, 기본적인 금융 용어조차도 잘 모르는 단계다. 금융 용어에 대한 기본 개념을 이해하는지 확인하는 질문이 위주이다. 예를 들어, 'ETF가 무엇인가요?', '기초지수가 무엇인가요?'와 같은 질문을 한다. 사용자층에는 매우 기초적인 용어와 개념을 중심으로 설명이 제공된다.

Learner는 기본적인 금융 용어를 들어본 적은 있지만, 정확한 의미나 적용에 대해서는

¹⁰ '농협은행, 개인맞춤형 'AI 금융상품 추천서비스' 출시 - IT조선 2024.06.13

확신이 없는 수준이다. 특정 금융 상품에 대한 관련 개념을 제대로 이해했는지 확인하거나 특정 부분을 확인하는 질문을 위주로 한다. 예를 들어, 'ETF는 배당을 언제 주나요?', '해당 상품의 세금 계산은 어떻게 하나요?'와 같은 질문을 한다. 이들에게는 이미 알고 있는 개념을 바탕으로 조금 더 심화된 설명을 제공하되, 복잡한 내용은 단계적으로 풀어 설명한다.

마지막으로 Investigator는 상당한 금융 지식을 갖춘 사용자로, 특정 금융 상품이나 최신 시장 동향에 대한 구체적인 정보를 요구하는 사용자층으로 구분한다. 질문의 예시는 '해당 ETF는 환헤지인가요?', '리밸런싱 주기는 어떻게 되나요?'와 같이 상대적으로 어려운 금융 용어를 활용하며 구체적인 정보를 요구한다.

이러한 단계별 세분화는 금융권에 진입하지 못했거나 금융에 익숙하지 않은 사용자들도 쉽게 접근할 수 있도록 설계되었으며, 이들이 부담 없이 이용할 수 있는 진입점을 제공한다. 동시에 충분한 지식을 갖춘 사용자들의 깊이 있는 정보 요구도 충족시킨다. 특히 Newcomer와 Learner 단계에 더 많은 초점을 맞추므로써, 금융 지식이 부족한 사용자들의 금융에 대한 심리적 장벽을 낮춘다.

2) 사용자의 질문 수준을 통한 사용자의 금융 지식 파악

질문의 구체성, 용어 사용, 연관성 등에는 그 사람이 해당 분야에 대한 이해도나 경험이 반영된다. 예를 들어, 초급자는 '적금과 예금의 차이는 무엇인가요?'라는 기초적인 질문을 하는 반면, 금융에 대한 높은 지식을 보유한 사람은 'ETF의 리스크 관리 방법에 대해 알려주세요.'와 같은 질문을 한다.

사용자의 금융 지식을 판단하기 위해 사전 테스트를 진행하는 기존 서비스와 달리, 본 아이디어는 위 질문 특성을 반영한다. 미리 정의한 금융 지식 수준 스케일에 맞춰 사용자의 질문을 통해 자동으로 질문에 대한 수준을 판별한다. 이후 누적된 질문들에 대한 수준을 바탕으로 사용자의 금융 지식 파악이 고도화된다.

이를 통해 사전 테스트 후 사용자의 질문에 대한 답변을 하는 것이 아닌, 바로 사용자의 질문에 대한 답변을 하면서 사용자의 수준을 판단하여, 사용자의 편의성을 높인다.

3) 개별 사용자의 금융 지식 수준을 고려한 답변

사용자마다 다른 금융 지식 수준을 파악하고, 그에 맞춘 설명을 제공하는 것을 목표로 한다.

초보자는 기본적인 용어와 개념을 이해하기 쉽도록 단순하고 직관적인 설명을 제공받고, 중급자나 전문가들은 보다 심화된 금융 정보와 분석을 통해 자신이 원하는 깊이 있는 정보를 얻을 수 있다. 또한 LLM 모델의 특성상, 이전 대화 맥락을 기억하고 이를 기반으로 답변의 깊이와 내용을 실시간으로 조정하는 특징이 있다.

이를 통해 각기 다른 사용자가 개인의 학습 속도와 필요에 맞춰 금융 용어 및 금융 상품에 대한 개념을 효과적으로 이해할 수 있다.

4) 보다 다양한 질문 답변 가능

기존에 제시된 서비스와 달리, 미리 정해진 금융 용어에 대한 설명만 제공하는 것이 아니라, 사용자가 궁금해하는 다양한 금융 질문에 대한 답변할 수 있는 확장성이 있다.

특정금융 용어에 대한 정의를 넘어서, 실제 금융 상품에 대한 이해 및 투자 전략 등 보다 구체적인 질문을 직접 할 수 있어, 사용자가 실질적인 금융적 도움을 얻을 수 있다.

5) 사용자 중심적 접근 서비스

본 서비스는 사용자가 별도의 프로필 작성이나 금융 수준을 사전에 입력하지 않아도 되며 로그인과 동의 절차만 진행하면 즉시 챗봇과의 대화가 가능하다. 그리고 사용자가 챗봇에 입력하는 질문이 곧 금융 지식 수준을 나타내는 데이터가 되어, 실시간으로 금융 지식 수준을 분석하고, 이에 맞춘 맞춤형 답변을 제공한다.

또한 다양한 예시와 시나리오를 통해 금융 개념을 더욱 쉽게 이해하도록 도와줄 수 있다. 생성형AI를 통해 1차적으로 답변을 하였지만, 사용자가 이해가 가지 않는 경우, '다른 예시로 알려줄래?' 등과 같이 추가적인 질문을 할 수 있어 사용자가 금융 지식 및 금융상품을 보다 쉽게 습득할 수 있도록 지원한다.

이는 사용자의 호기심과 학습 욕구를 최우선으로 존중하는 진정한 의미의 사용자 중심 서비스를 제공한다.

이러한 본 기획서의 접근 방식은 사용자의 금융 상품 진입 장벽을 최소화하고, 금융 관련지식 습득에 대한 심리적 장벽을 크게 낮출 수 있다. 또한 사용자가 자신의 페이스에 맞춰 금융 지식을 확장해 나갈 수 있도록 지원함으로써, 장기적으로 더 깊이 있는 금융 이해와 현명한 금융 상품 결정을 촉진할 수 있다.

5. 아이디어의 실현가능성

본 아이디어는 사용자의 질문을 통해 금융 지식 수준을 파악하고, 사용자별 맞춤형 답변을 목표로 한다. 이를 위해 사용되는 핵심적인 기술은 생성형AI인 LLM(Large Language Model)과 RAG(Retrieval-Augmented Generation)기술이다. 본 아이디어의 PoC 단계에서는 OpenAI의 GPT-4o-mini와 Ragas, 랭체인 등을 활용하여 구현한다.

5.1 LLM을 통한 사용자 금융 지식 수준 맞춤형 답변 생성

1) 사용자의 금융지식 수준 수준 판단

언어 이해 능력 및 논리적 추론 능력이 뛰어난 LLM을 활용하여 사용자의 질문을 이해한다. 그 후 사용자의 질문에 대해 few-shot learning으로 'Newcomer', 'Learner',

'Investigator' 중 한 가지로 질문 수준을 판단한다. Few-shot learning은 LLM모델의 In-Context Learning 관련 특징 중 하나로, 모델이 매우 제한된 양의 학습 데이터로 작업을 하는 학습 방식이다. 본 아이디어에서는 'Newcomer', 'Learner', 'Investigator'의 수준에서 물을법한 질문을 사전에 3~5가지로 정의한 뒤 LLM모델의 프롬프트에 입력한다. 이를 통해 LLM 모델이 사용자 질문 수준 판단을 3-shot, 5-shot learning을 하도록 한다.

2) 사용자의 질문에 해당되는 답변 생성

사용자의 질문에 대한 답변은 RAG를 통해 얻어진 내용을 바탕으로 답변을 생성한다. 사용자가 금융 지식 및 특정 금융 상품 등 다양한 질문을 하더라도, LLM의 언어 처리 능력을 통해 답변할 수 있다. 또한 기존 대화 이력을 바탕으로 LLM이 사용자의 관심있는 부분의 질문을 파악하여 추후 질문에 대한 답변에서도 이를 반영하여 생성할 수 있다.

3) 사용자의 금융 지식 수준 맞춤형 답변 생성

2번에서 생성한 답변을 바탕으로, 사용자의 누적된 질문을 통해 측정된 금융 지식 수준에 맞게 가공하여 답변한다. 예를 들어, 사용자의 질문 이력이 'Newcomer': 4개, 'Learner': 1개, 'Investigator': 0개로 측정되었다면, 비율이 각각 80%, 20%, 0%이다. LLM이 답변을 생성할 때, 프롬프트에 해당 비율을 알려주며 '사용자의 수준을 고려한 답변을 생성해줘'라고 기입한다. 언어 이해 및 추론 능력이 뛰어난 LLM은 해당 프롬프트를 바탕으로 사용자의 금융 지식 수준을 고려한 답변을 가공한다.

5.2 RAG를 통한 답변 정확성 향상

생성AI인 LLM의 주요 취약점 중 하나는 환각(Hallucination)이 있다는 점이다. 이를 억제하고자 검색 증강 생성 기술인 RAG를 통해 LLM이 신뢰성 있는 답변을 생성하도록 돕는다. RAG는 LLM의 학습 데이터셋 이외인 외부 데이터에서 관련 정보를 효율적으로 검색한다. 또한 외부 데이터를 업데이트 함으로써 최신성을 유지할 수 있다는 장점도 있다.

사용자의 질문에 대해 LLM 모델이 정확한 답변을 했는지 판단하기 위해, RAG 평가용 LLM으로 판단한다. 성능 지표는 Faithfulness¹¹, Answer_Relevancy¹², Context_Precision¹³, Context_Recall¹⁴ 평가를 수행한다.

Faithfulness는 주어진 컨텍스트에 대해 생성된 답변이 사실적 일관성이 있는지를 판단한다. 자세한 수식은 아래 그림과 같다.

¹¹ <https://docs.ragas.io/en/stable/concepts/metrics/faithfulness.html#equation-faithfulness>

¹² https://docs.ragas.io/en/v0.0.19/concepts/metrics/answer_relevance.html#answer-relevance

¹³ https://docs.ragas.io/en/v0.0.19/concepts/metrics/context_precision.html#context-precision

¹⁴ https://docs.ragas.io/en/v0.0.19/concepts/metrics/context_recall.html#context-recall

$$\text{Faithfulness score} = \frac{|\text{Number of claims in the generated answer that can be inferred from given context}|}{|\text{Total number of claims in the generated answer}|}$$

Answer_Relevance는 답변 관련성으로, 주어진 질문에 대해 생성된 답변이 얼마나 적절하고 관련성이 있는지를 평가하는데 중점을 둔다.

Context_Precision은 문맥 정밀도로, 문맥 내에서 실제로 관련 있는 항목들이 모두 상위 순위에 매겨졌는지를 평가한다. 자세한 수식은 아래 그림과 같다. (k는 문맥내 chunks의 전체 개수이다.)

$$\text{Context Precision@k} = \frac{\sum \text{precision@k}}{\text{total number of relevant items in the top K results}}$$

$$\text{Precision@k} = \frac{\text{true positives@k}}{(\text{true positives@k} + \text{false positives@k})}$$

Context_Recall은 문맥 재현율로, 검색된 문맥이 정답으로 간주되는 답변과 얼마나 일치하는지를 측정한다. 자세한 수식은 아래 그림과 같다.

$$\text{context recall} = \frac{|\text{GT sentences that can be attributed to context}|}{|\text{Number of sentences in GT}|}$$

위 네가지 지표를 활용하여 기존 정의한 데이터셋에서 LLM 모델이 RAG를 통해 적절한 답변을 생성하였는지 평가하고, 이를 바탕으로 모델 성능을 개선한다.

6. 아이디어의 문제 해결 가능성

6.1 현재 문제를 해결하기 위한 아이디어 활용 방안

본 아이디어는 금융 지식 수준에 맞춘 맞춤형 설명을 통해 두 가지 주요 문제를 해결하고자 한다.

1) 금융 지식 격차를 해소할 수 있다.

현재 대한민국의 금융 교육은 전반적으로 미흡한 수준이며, 많은 사람들이 금융 용어와 상품을 제대로 이해하지 못하고 있다 (2.아이디어 구상 및 제안 배경 참고).

본 서비스는 복잡한 금융 용어를 사용자가 쉽게 이해할 수 있도록 맞춤형 설명을 제공하며, 나이대와 자산 수준과는 상관없이 사용자가 자신만의 속도로 금융 지식을 확장할 수 있게 돕는다. 이를 통해 금융 용어에 대한 막연한 두려움을 없애고, 금융 시장에 대한 접근성을 높임으로써 건전한 금융 생활을 영위하는 데 중요한 기반을 제공할 수 있다.

2) 금융 상품 가입 장벽을 완화한다.

현재 많은 개인이 금융 상품에 접근할 때 정보 부족과 복잡한 용어로 인해 가입을 망설이는 경우가 있다. 특히, 금융 투자 시장에서는 소액 투자자나 초보 투자자는 전문가들이 사용하는 금융 용어와 개념에 대한 이해 부족으로 인해 투자 결정을 내리는 데 어려움을

겪고 있다.

본 서비스는 금융 상품에 대한 사용자 맞춤형 정보를 제공함으로써 이러한 장벽을 낮추고자 한다. 금융 지식 수준에 맞춘 설명을 통해 더 쉽게 이해하고 자신감을 가지고, 본인에게 맞는 상품 가입을 결정을 내릴 수 있도록 돕는다. 이를 통해 사용자는 자신에게 적합한 금융 상품을 찾고, 보다 쉽게 소액 투자에 대한 지식을 얻을 수 있다.

이러한 문제 해결 방식을 통해 본 기획서는 습득한 금융 지식을 바탕으로, 개인의 금융 건강성과 금융 시장 접근성을 향상시키는 것은 물론, 장기적으로는 금융 서비스의 보편화를 촉진하고 금융 지식 격차를 완화할 수 있다.

본 아이디어는 사용자의 금융 지식 및 금융 상품에 대한 이해도를 높이는 것을 목표로 한다. 이를 위해 사용자의 질문을 통해 금융 지식을 파악하고, 수준별 맞춤형 답변을 제시한다. 제시된 아이디어를 통해 아래와 같이, 사용자의 금융 지식 수준에 따른 맞춤형 답변을 전달할 수 있다.

6.2 아이디어 활용 적용 예시: 사용자 금융 지식 수준에 따른 맞춤형 답변 비교

사용자의 질문을 Newcomer, Learner, Investigator 3단계 레벨로 분류한다. 이후 누적되는 각 레벨 비율을 바탕으로 사용자의 금융 지식 수준을 판단한다.

동일 질문이라도 사용자의 금융 지식에 따라 맞춤형 답변을 생성하는지 확인하기 위해, 아래와 같은 실험을 비교한다.

PoC 제출파일 중 chat_logs 폴더에 저장된 Newcomer_synario, Learner_synario, Investigator_synario로 저장된 csv 파일이 있다. 해당 파일에는 각 레벨의 사용자가 물었던 질문 이력들과 RAG를 통해 생성된 답변, 사용자의 금융 지식 수준에 따라 정제된 답변, 사용자의 현재 금융 지식 수준 등이 담겨 있다. 총 5번의 질문 중 4번째까지는 각 레벨의 사용자가 묻는 질문들로 유형화하였고, 이후 5번째 질문에는 ‘ETF가 무엇인지 개념설명해줘’ 라는 질문으로 동일하게 구성하였다.

시나리오 기록 파일과 같이, 동일한 질문 내용을 토대로 RAG 답변이 생성되었다. QA모델을 통해 나온 답변은 동일 질문이여도 문법 및 조사 등 약간은 다른 답변이지만, 주요 키워드 및 답변의 맥락 및 설명은 동일하게 나온다. 아래는 사용자의 현재 금융 지식 수준에 맞게 가공된 전후 답변이 다른 양상임을 확인할 수 있다.

1) Newcomer에는 Learner 및 Investigator보다 ‘여러 가지 자산에 투자할 수 있다’ , ‘돈을 찾을 수 있다’ 등 직관적인 용어 및 쉬운 개념 풀이로 답변이 된다. 특히, 사용자 수준 맞춤형 가공 전 답변에서는 ‘유동성’ 을 ‘주석처럼 거래 가능’ 과 같이 더 쉬

운 개념으로 바뀌서 설명함을 알 수 있다.

표 3. Newcomer의 금융 수준 맞춤형 가공 전후 답변 비교

사용자1: Newcomer QA 답변 가공 전
ETF(상장지수펀드, Exchange-Traded Fund)는 주식시장에 상장되어 거래되는 펀드의 일종으로, 특정 지수나 자산군을 추종하는 투자 상품입니다. ETF는 여러 자산에 분산 투자할 수 있는 장점을 제공하며, 주식처럼 실시간으로 매매가 가능합니다. ETF의 주요 특징은 다음과 같습니다: 1. 다양한 자산군: ETF는 주식, 채권, 원자재 등 다양한 자산에 투자할 수 있습니다. 2. 유동성: 주식처럼 거래소에서 실시간으로 사고팔 수 있어 유동성이 높습니다. 3. 비용 효율성: 일반적으로 관리 수수료가 낮아 비용 효율적인 투자 방법으로 여겨집니다. 4. 투자 접근성: 소액으로도 다양한 자산에 투자할 수 있어 개인 투자자에게 접근성이 좋습니다. 하지만 ETF는 실적 배당 상품으로, 운용 결과에 따른 이익 또는 손실이 투자자에게 귀속되며, 원본 손실이 발생할 수 있다는 점을 유의해야 합니다. 또한, 과거의 운용 실적이 미래의 수익을 보장하지 않으므로, 투자 전에 충분한 정보와 설명서를 읽는 것이 중요합니다.
사용자1 Newcomer {Newcomer: 80%, Learner: 20%, Investigator: 0} QA답변 가공 후
ETF는 "Exchange-Traded Fund"의 약자로, 상장지수펀드라고도 부릅니다. 쉽게 말해, ETF는 주식처럼 거래소에서 사고팔 수 있는 특별한 유형의 펀드입니다. 이 펀드는 여러 주식이나 다른 자산을 모아서 만든 것이기 때문에, ETF를 구매하면 여러 자산에 동시에 투자하는 효과를 얻을 수 있습니다. ETF의 주요 특징은 다음과 같습니다: 1. 다양한 자산에 투자: ETF는 주식, 채권, 원자재 등 여러 가지 자산에 투자할 수 있습니다. 2. 주식처럼 거래 가능: 주식처럼 실시간으로 사고팔 수 있어, 필요할 때 쉽게 돈을 찾을 수 있습니다. 3. 비용 효율적: 관리 수수료가 상대적으로 낮아, 투자 비용을 줄일 수 있습니다. 4. 투자 분산: 하나의 ETF에 투자함으로써 여러 자산에 분산 투자할 수 있어 리스크를 줄이는데 도움이 됩니다. 이렇게 투자하면 ETF의 수익이나 손실이 직접적으로 투자자에게 돌아옵니다. 하지만 모든 투자는 위험이 따르므로, 투자하기 전에 관련된 내용과 설명서를 잘 읽어보는 것이 중요합니다.

2) Learner에는 Newcomer에는 없는 ‘유동성’이라는 키워드가 그대로 반영되며, ‘이익이나 손실이 투자자에게 귀속된다’, ‘쉽게 자산을 매도할 수 있다’ 등 기본적인 금융 용어 개념을 바탕으로 답변이 된다.

표 4 Learner의 금융 수준 맞춤형 가공 전후 답변 비교

사용자2: Learner QA 답변 가공 전

ETF(상장지수펀드, Exchange-Traded Fund)는 주식시장에 상장되어 거래되는 펀드로, 특정 지수나 자산군의 성과를 추적하는 투자 상품입니다. ETF는 여러 자산에 분산 투자할 수 있는 장점을 제공하며, 주식처럼 거래소에서 실시간으로 매매할 수 있습니다.

ETF는 일반적으로 다음과 같은 특징을 가지고 있습니다:

1. 다양한 자산군: ETF는 주식, 채권, 원자재 등 다양한 자산에 투자할 수 있습니다.
2. 실적 배당: ETF는 운용 결과에 따라 발생하는 이익이나 손실이 투자자에게 귀속됩니다.
3. 유동성: 주식처럼 거래소에서 실시간으로 사고팔 수 있어 유동성이 높습니다.
4. 비용 효율성: 일반적으로 관리 수수료가 낮아 비용 효율적인 투자 방법으로 여겨집니다.
5. 투자 설명서: ETF를 취득하기 전에 투자 대상, 환매 방법, 보수 등에 대한 정보를 담고 있는 투자 설명서를 반드시 읽어야 합니다.

하지만 ETF는 원본 손실이 발생할 수 있으며, 과거의 운용 실적이 미래의 수익을 보장하지 않는다는 점도 유의해야 합니다.

사용자2 Learner {Newcomer: 0, Learner: 100%, Investigator: 0} QA답변 가공 후

ETF(상장지수펀드, Exchange-Traded Fund)는 주식시장에 상장되어 거래되는 펀드로, 특정 지수나 자산군의 성과를 추적하는 투자 상품입니다. ETF는 다양한 자산에 분산 투자할 수 있는 장점을 제공하며, 주식처럼 거래소에서 실시간으로 매매할 수 있습니다. 이와 관련하여 몇 가지 중요한 특징을 살펴보면:

1. 다양한 자산군: ETF는 주식, 채권, 원자재 등 여러 자산에 투자할 수 있어 **특정 산업이나 지역에 대한 리스크를 줄이는 데 도움이 됩니다.**
2. 실적 배당: ETF가 투자하는 기초자산의 성과에 따라 발생하는 **이익이나 손실이 투자자에게 귀속됩니다.** 예를 들어, ETF가 포함한 기업들이 배당금을 지급하면, 그 배당이 ETF에도 반영되어 투자자가 배당을 받을 수 있습니다.
3. **유동성:** ETF는 주식처럼 거래소에서 실시간으로 사고팔 수 있어 **매우 유동적입니다. 이는 투자자가 필요할 때 쉽게 자산을 매도할 수 있는 장점이 있습니다.**
4. 비용 효율성: 일반적으로 ETF는 관리 수수료가 낮아, 투자자에게 더 유리한 조건을 제공합니다. 따라서 **장기 투자 시 비용 절감 효과를 누릴 수 있습니다.**
5. 투자 설명서: ETF를 구매하기 전에는 반드시 투자 설명서를 검토해야 합니다. 이 설명서에는 투자 대상, 환매 방법, 수수료 및 리스크 정보가 포함되어 있어 투자 결정을 하는 데 중요한 역할을 합니다.

마지막으로, ETF에도 투자 리스크가 존재하므로, 원본 손실이 발생할 수 있으며 과거의 운용 실적이 미래의 수익을 보장하지 않는 점은 유의해야 합니다.

3) Investigator에는 ‘포트폴리오 다각화’, ‘실적 배당 상품’ 등 Newcomer, Investigator 레벨보다 더 어려운 금융 용어를 활용하여 설명해줄을 확인할 수 있다. 특히, ‘chat_logs’ 폴더의 Investigator 질문 이력에서 ‘TIGER 글로벌AI액티브 ETF에 투자했을 때의 세금 계산 방식에 대하여 설명해줘.’를 4번째로 사용자가 질문하였다. Newcomer, Investigator에서의 예시와 다르게, 5번째 질문에서는 이를 고려하여, ‘ETF가 무엇인지에 대한 설명에 ‘세금 측면에서도 거래세가 면제되는 경우가 있다’와 같이 사

용자의 이전 질문을 기억해, 사용자의 관심사를 반영하여 답변함을 확인할 수 있다.

표 5. Investigator의 금융 수준 맞춤형 가공 전후 답변 비교

사용자3: Investigator QA 답변 가공 전
ETF(상장지수펀드, Exchange-Traded Fund)는 주식시장에 상장되어 거래되는 펀드의 일종으로, 특정 지수나 자산군을 추종하는 투자 상품입니다. ETF는 여러 자산에 분산 투자할 수 있는 장점을 제공하며, 주식처럼 실시간으로 매매가 가능합니다. ETF의 주요 특징은 다음과 같습니다: 1. 다양한 자산군: ETF는 주식, 채권, 원자재 등 다양한 자산에 투자할 수 있습니다. 2. 유동성: 주식처럼 거래소에서 실시간으로 사고팔 수 있어 유동성이 높습니다. 3. 비용 효율성: 일반적으로 관리 수수료가 낮아 비용 효율적인 투자 방법으로 여겨집니다. 4. 투자 접근성: 소액으로도 다양한 자산에 투자할 수 있어 개인 투자자에게 접근성이 좋습니다. 하지만 ETF는 실적 배당 상품으로, 운용 결과에 따른 이익 또는 손실이 투자자에게 귀속되며, 원본 손실이 발생할 수 있다는 점을 유의해야 합니다. 또한, 과거의 운용 실적이 미래 수익을 보장하지 않으므로, 투자 전에 충분한 정보와 설명서를 읽는 것이 중요합니다.
사용자3 Investigator{Newcomer: 0, Learner: 0, Investigator: 100%} QA답변 가공 후
ETF(상장지수펀드, Exchange-Traded Fund)는 주식시장에 상장되어 거래되는 펀드로, 특정 지수나 자산군 (예: 주식, 채권, 원자재 등)을 추종하는 투자 상품입니다. ETF는 여러 자산에 분산 투자할 수 있는 장점을 제공하며, 주식처럼 실시간으로 매매가 가능합니다. ETF의 주요 특징은 다음과 같습니다: 1. 다양한 자산군: ETF는 주식, 채권, 원자재 등 여러 자산에 투자할 수 있어 포트폴리오 다각화에 유리합니다. 2. 유동성: 주식처럼 거래소에서 실시간으로 사고팔 수 있어 유동성이 높아 필요할 때 쉽게 매매할 수 있습니다. 3. 비용 효율성: 관리 수수료가 일반적으로 낮고, 세금 측면에서도 거래세가 면제되는 경우가 있어 비용 부담이 적습니다. 4. 투자 접근성: 소액으로도 다양한 자산에 투자할 수 있어 개인 투자자에게 보다 쉽게 접근할 수 있는 투자 수단입니다. 5. 실적 배당 상품: ETF는 투자자의 운용 성과에 따라 이익 또는 손실이 발생하며, 원본 손실이 생길 수 있다는 점을 유의해야 합니다. 과거의 운용 실적이 미래 수익을 보장하지 않기 때문에 투자 전에 충분한 정보와 설명서를 읽는 것이 중요합니다. 따라서, ETF는 다양한 투자 전략을 활용할 수 있는 유용한 투자 상품이며, 각 ETF의 특성 및 구조에 대한 이해가 필요합니다. 추가적인 질문이나 더 구체적인 정보가 필요하시면 언제든지 문의해 주세요.

이러한 사용자별 금융 지식 맞춤에 따른 답변은 아래 표와 같은 차이를 보인다. 해당 예시는 표 3, 4, 5의 예시 중 일부분이다. 가공 전 동일한 항목에 대해, 가공 후의 Newcomer, Learner, Investigator의 수준별 답변이 서로 다를 수 있다. 각 사용자의 수준을 고려하여 Newcomer에게는 직관적인 효과를 설명하고, Learner에게는 더 장기적인 측면에서의 설명을, Investigator에는 ‘포트폴리오 다각화’와 같은 상대적으로 어

려운 용어를 활용하여 부연 설명함을 알 수 있다.

표 6. 동일 답변에 대해 사용자의 수준에 따른 답변 양상 비교

특징	답변 비교
가공 전 답변	1. 다양한 자산군: ETF는 주식, 채권, 원자재 등 다양한 자산에 투자할 수 있습니다.
Newcomer 가공 후	1. 다양한 자산에 투자: ETF는 주식, 채권, 원자재 등 여러 가지 자산에 투자할 수 있습니다.
Learner 가공 후	1. 다양한 자산군: ETF는 주식, 채권, 원자재 등 여러 자산에 투자할 수 있어 특정 산업이나 지역에 대한 리스크를 줄이는 데 도움이 됩니다.
Investigator 가공 후	1. 다양한 자산군: ETF는 주식, 채권, 원자재 등 여러 자산에 투자할 수 있어 포트폴리오 다각화에 유리합니다.

위 실험을 통해 동일한 질문이라도 사용자의 금융 지식에 따라 맞춤형 답변을 생성하는 것을 알 수 있으며, 이러한 방식은 사용자 중심의 접근으로, 사용자가 기존 설명보다 더 쉽게 이해하고 받아들일 것으로 기대된다.

6.3 고려되는 리스크 및 문제 해결 방안

1) 사용자의 금융 지식 수준 구분 모호성

아이디어 구현은 RAG를 통해 생성된 답변을 토대로, 프롬프트와 LLM 모델을 거쳐 사용자의 금융 지식 수준 파악 및 가공된 답변을 제시한다. LLM 1개가 사용자 금융 지식 수준을 분류하는 태스크와 답변을 가공하는 태스크, 2개를 동시에 수행하고 있다. 또한 프롬프트를 통해 의도한 결과를 도출하려고 할 때 이전 대화 이력이나 사용자의 투자 수준의 비중이 고려된다.

위 점들을 종합하였을 때, 사용자의 금융 지식 수준을 평가하는 과정에서 질문의 순서나 질문의 어조 등에 따라 사용자 투자 수준이 달라지는 현상을 확인하였다.

표 7. 동일 질문이여도 이전 대화 이력 등에 따라 달라지는 모델의 금융 지식 수준 판단

 ETF의 개념에 대해서 설명해줘.  위 질문으로 판단한 고객 투자 수준: Newcomer 답변: ETF는 주식처럼 거래소에서 사고팔 수 있는 투자 상품입니다. 주로 특정 시장 지수(예: 코스피 또는 나스닥 지수)의 움직임을 따라가도록 설계되어 있습니다. 쉽게 말해, ETF를 사면 그 지수를 추종하는 여러 주식을 한 번에 구매하는 것과 비슷합니다. ETF는 장기 투자나 단기 거래 모두에 적합하며, 다양한 자산에 분산 투자할 수 있는 장점이 있습니다.	 ETF는 배당을 언제 주는지에 대해서 설명해줘.  위 질문으로 판단한 고객 투자 수준: Learner 답변: ETF가 투자하는 기초자산의 종류에 따라 각 ETF별로 배당정책이 결정됩니다. 일반적으로 국내주식형 ETF는 매월 말 또는 분기말을 기준으로 배당금을 지급합니다. 만약 ETF가 채권에 투자하고 있다면, 보유하고 있는 채권의 이자지급 시점을 고려하여 배당금이 지급되며 보통 매 분기마다 정기적으로 지급됩니다. 이를 통해 투자자들은 ETF를 통해 기초 자산에서 발생하는 수익을 정기적으로 받을 수 있습니다.
 ETF의 개념에 대해서 설명해줘.  위 질문으로 판단한 고객 투자 수준: Learner 답변: ETF는 Exchange-Traded Fund의 약어로, 특정 지수나 자산의 수익률을 추종하는 펀드입니다. 즉, ETF는 주식처럼 거래소에서 매매할 수 있는 투자 상품으로, 일반적으로 다양한 주식이나 채권 등을 포함하고 있습니다. ETF는 보통 특정 시장 지수(예: S&P 500, 코스피 등)의 성과를 따라가도록 만들어지며, 투자자는 ETF를 통해 여러 자산에 동시에 투자할 수 있는 장점을 가집니다. 또한, ETF는 주식과 유사하게 실시간으로 거래가 가능하므로, 투자자는 시장 상황에 따라 즉시 매매를 할 수 있습니다.	

이를 해결하기 위해서는 기존의 1개의 LLM을 통해 사용자의 금융 지식 수준 구분 및 답변 가공 작업을 수행하는 방식에서 벗어나, 사용자 금융 지식 수준 구분을 위한 LLM과 이를 토대로 RAG 답변을 가공하는 LLM을 구분한다. 이를 통해 각 LLM 모델 1개에 더 구체적인 역할을 부여하여 문제를 해결할 것으로 기대한다. 하단의 구조도와 같이 설계를 하여 문제를 해결할 수 있다.

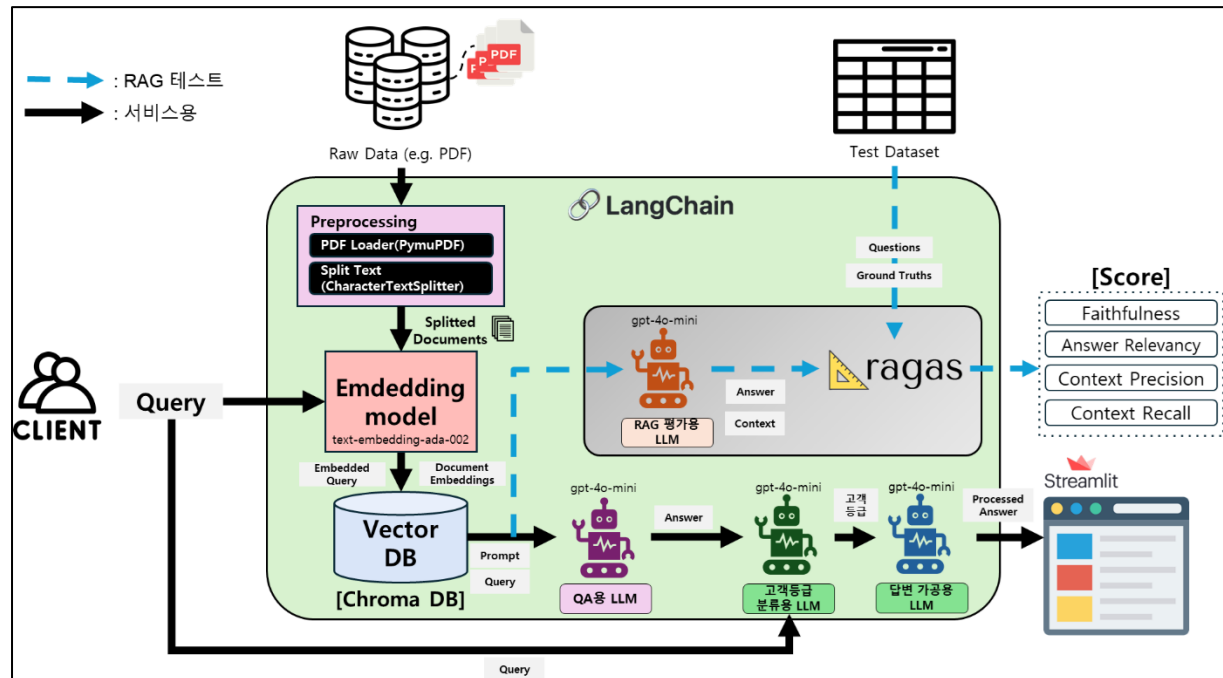


그림 8. LLM 모델 3개를 활용했을 때의 구조도

뿐만 아니라, few-shot-prompting 기법을 사용할 때, 사용자 경험을 거쳐 더 풍부한 예시를 추가하거나, 각 사용자의 금융 수준별 질문 데이터셋을 구축하여 모델에 fine-tuning 하는 등의 방법으로 해결할 수도 있다.

2) QA용 LLM의 고도화된 금융 지식 필요성

사용자에게 올바른 금융 지식을 전달하기 위해서는, QA용 LLM이 풍부한 금융 지식을 갖추고 있어야 한다.

이를 위해 금융 도메인의 지식으로 pre-training된 LLM을 활용하거나, 추가 금융 지식을 Fine-tuning 하는 등을 통해 해결할 수 있다.

3) Open-API 활용시의 비용

현재 OpenAI의 gpt-4o-mini와 임베딩 모델로 PoC를 제시하였다. OpenAI의 모델 활용시 일정 금액의 비용을 지불해야하기에, 대규모 서비스에 적용시 적지 않은 비용이 소모될 수 있다. 또한 gpt-4o-mini보다 더 우수한 모델을 활용할 경우 해당 비용은 더 증가될 수 있다.

해당 문제를 해결하기 위해, 기업 내 자체 LLM을 구축하거나 OpenAI의 비용 대비 성능이 준수한 모델을 선택하는 등을 통해 비용 측면을 완화할 수 있을거라 기대한다.

4) 사용자의 악의적인 공격 및 모델의 위험한 답변 생성 가능성

생성형AI에는 사용자단에서 다양한 프롬프트 및 입력을 적용할 수 있다. 이러한 특성 때문에 사용자가 금융 지식에 대한 질문이 아닌, 악의적인 공격 및 모델의 부적절한 답변을 유도할 가능성도 있다.

이를 위해서는, AI 가드레일을 구축하여 금융 지식과 무관한 입력이 들어올 경우, 선제적으로 분류하여 대응하는 등의 방법이 적용될 수 있다.

5) 특정 투자 상품 유도 방지

본 아이디어의 목적은 사용자의 금융 이해도를 높이는 것이다. 사용자가 특정 상품에 대한 질문을 한다면 답변을 하지만, 해당 상품을 구매하도록 유도하는 답변을 생성하지 말아야 한다.

이를 위해서는 서비스에 ‘금융 상품에 대한 구매는 사용자의 판단 하에 진행할 것을 권고 드립니다.’ 라고 직접적으로 명시하거나, 모델의 프롬프트에 ‘구매를 유도하는 내용은 넣지 않아야 한다.’ 라는 등의 규칙을 적용할 수 있다.

7. 아이디어의 발전 가능성

본 아이디어는 사용자 맞춤형 AI 금융 지식 답변 서비스를 기반으로, 향후 수요성, 시장성, 사업화 면에서 발전 가능성을 지니고 있다.

7.1 수요성 측면

1) AI 기술의 고도화를 통한 사용자 경험을 혁신

생성AI를 활용하여 개별 사용자의 질문 패턴을 분석하고, 이를 기반으로 최적화된 답변을 제공할 수 있다. 자연어 처리 기술 및 검색 기술의 발전으로 챗봇의 대화형 학습 경험 이 더욱 자연스럽고 효과적으로 개선될 것이다.

사용자들은 다양한 금융 지식을 더욱 쉽고 빠르게 습득하고 싶어한다. 생성형AI를 통한 개인화된 경험은 사용자 중심의 접근 방법으로, 쉽게 금융 지식을 습득할 수 있는 환경을 제공한다. 이를 통해 사용자가 기대하는 서비스의 본질적인 부분인 금융 학습 욕구를 충족시킬 수 있다.

7.2 시장성 측면

1) 실시간 금융 데이터 통합 시스템을 구축

최신 금융 데이터를 사용자에게 제공할 수 있다. 외부 API를 통해 실시간 데이터를 통합

함으로써, 사용자가 보다 실질적이고 정확한 금융 의사결정을 내릴 수 있도록 지원한다.

2) 금융 기관과의 협력을 통한 신뢰도 및 시너지 효과 강화

주요 금융 기관과 파트너십을 구축함으로써 플랫폼에 해당 기관의 금융 상품 정보를 통합하고, 이를 통해 신뢰할 수 있는 데이터와 정확한 정보를 제공할 수 있다. 이러한 협력은 사용자에게 더 나은 경험을 제공할 뿐만 아니라, 금융 기관 측면에서도 플랫폼을 통해 자사 상품을 더욱 효과적으로 홍보하고, 잠재 사용자를 확보할 수 있는 기회를 제공한다. 예를 들어, 특정 은행과의 협력을 통해 그 은행의 금융 상품을 플랫폼에 반영함으로써, 사용자들은 다양한 금융 상품을 쉽게 비교하고 선택할 수 있을 뿐만 아니라, 금융 기관은 이를 통해 사용자 유입과 상품 판매를 촉진할 수 있다.

금융 기관과 협력했을 때의, 각 사용자의 로그인 기능을 담은 B2C 서비스 프로세스 플로우차트는 아래 그림과 같다. 예를 들어, 고객의 금융 지식 수준이라는 민감정보를 처리하기 위한 필수 약관 동의와 관련된 부가적인 설명이나, 동의하지 않더라도 RAG 답변에 의해 정확한 답변을 제공할 수 있다는 점을 명시한다.

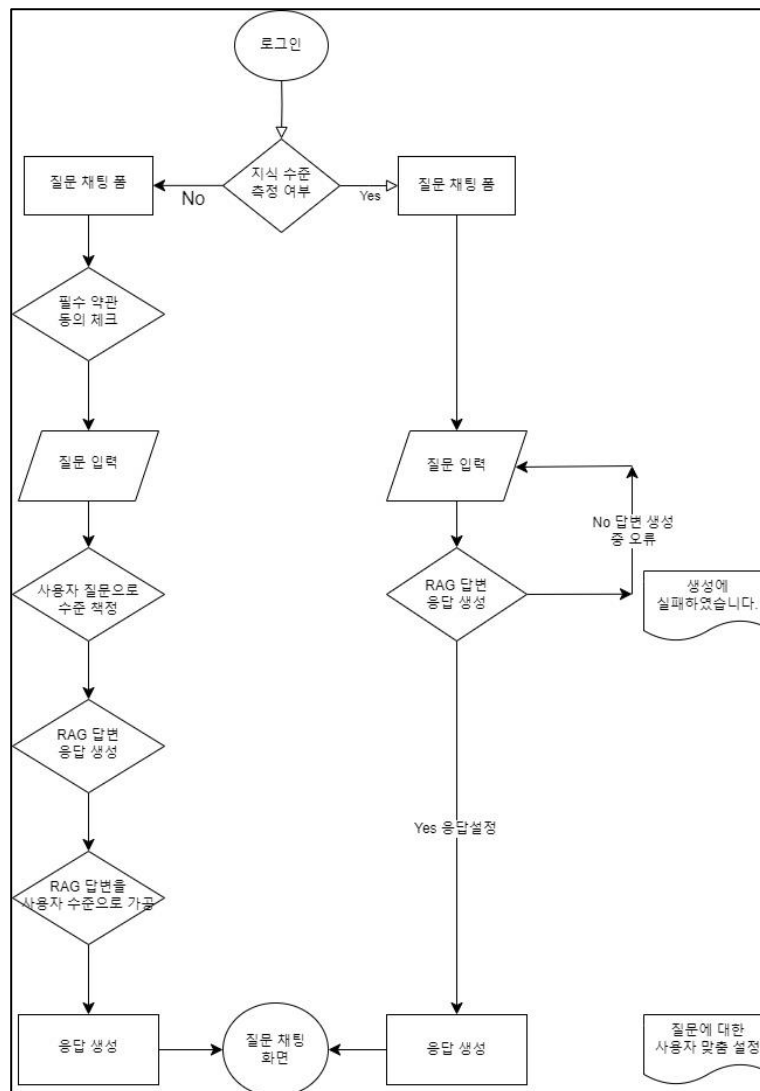


그림 9. 금융기관과 협력한 B2C 서비스 프로세스 플로우차트 예시

7.3 사업화 가능성

1) 사용자 친화적인 UI/UX 개발

다양한 연령대와 금융 지식 수준에 맞춘 맞춤형 인터페이스와 대화 스타일을 제공한다. 예를 들어, 초보자 단계(Newcomer)의 이용자를 위해서는 친근하고 캐주얼한 스타일의 캐릭터를, 중급자 단계(Investigator)의 이용자를 위해서는 신뢰감 있는 전문가 이미지의 챗봇을 도입하여 사용자 특성에 맞는 챗봇 컨셉 및 인터페이스를 구현할 수 있다. 이를 통해 더 많은 사용자 확보와 유지가 가능할 수 있다.

2) 개인 금융 상황 분석 및 맞춤형 조언 시스템을 통합

단순 정보 제공을 넘어 실질적인 금융 조언까지도 제공하는 진정한 사용자 맞춤형 금융 지식 서비스를 제공할 수 있다. 사용자의 동의를 얻어 개인의 금융 데이터를 분석하고, 이를 바탕으로 맞춤형 금융 조언을 제공하여 사용자는 자신의 상황에 가장 적합한 금융 의사결정을 내릴 수 있게 된다. 이는 금융 상담 서비스와 같은 수익화 모델로 직결될 가능성이 있다.

3) 단계별 맞춤형 금융 교육 서비스 확장 가능성

기존 사용자 데이터를 바탕으로, 금융 지식이 부족한 사용자에게 단계별 교육 콘텐츠를 제공한다. 사용자는 자신의 금융 지식 수준에 맞춘 교육을 점진적으로 학습하며, 복잡한 금융 개념을 쉽게 이해할 수 있게 된다. 이를 통해 금융 지식이 부족한 사용자도 단계별로 차근차근 이해도를 높이며, 점점 더 복잡한 개념을 쉽게 받아들일 수 있다. 단순한 정보 제공을 넘어 금융 이해력을 체계적으로 향상시키는 교육 플랫폼으로 발전해, 사용자들이 스스로 금융 지식을 쌓고, 더 나은 금융 의사결정을 내릴 수 있도록 돕는 역할을 수행할 수 있다. 이는 교육 프로그램, 유료 구독 서비스와 같은 수익화 모델로 도입할 수 있다.

이러한 발전 방향들을 통해, 본 서비스는 사용자의 금융 활동의 진입 장벽을 낮추고, 사용자에게 맞춤형 금융 지식을 제공함으로써, 더 많은 사람들이 효과적으로 금융 활동을 시작할 수 있도록 도울 수 있다.

8. 아이디어의 적용 대상 및 효과

8.1 아이디어의 적용 대상

본 서비스의 주요 적용 대상은 금융 지식(금융 이해력)이 낮은 사용자들이다. 금융 지식이 낮은 고령층도 해당되지만, 특히 경제활동을 막 시작하거나, 금융 상품에 관심을 갖기 시작한 20-30대 청년층이 많이 해당될 것으로 예상된다. 이 연령대는 직업을 구하고 경제적 독립을 시작하며, 자산 증식과 재정적 목표를 설정하는 시기로, 다양한 금융 상품에 대한 관심이 높아지는 시기이다. 예를 들어, 청년층은 기존 금융 교육의 부족으로 인해

금융 지식이 부족한 경우가 많아, ETF와 같은 투자 상품에 대한 이해가 부족하여 진입장벽을 느끼는 경우가 많다. 이들은 금융 용어와 개념의 복잡성 때문에 투자 결정을 미루거나 쉽게 포기할 가능성이 높다.

따라서, '사용자 맞춤형 AI 금융 지식 답변 서비스'는 이러한 금융 지식이 낮은 사용자들을 위해 설계되었다. 이 서비스는 사용자의 금융 지식 수준을 평가한 후, 맞춤형으로 이해하기 쉬운 용어로 관련 금융 지식 및 금융 상품에 대한 설명을 제공함으로써, 사용자들이 금융에 대한 자신감을 가지게 하고, 금융 상품에 대한 접근성을 높인다.

8.2 아이디어의 적용 효과

본 아이디어의 적용시 기대 효과는 아래 네 가지와 같다.

1) 금융 용어에 대한 이해도 증진

본 서비스는 사용자의 금융 지식 수준에 맞춘 맞춤형 설명을 제공함으로써, 복잡한 보험, 투자 용어 등을 쉽게 이해할 수 있도록 돕는다. 이를 통해 금융 지식이 낮은 사용자들이 금융 상품에 대해 명확한 이해를 가질 수 있게 되어, 금융 상품 선택 시 더 나은 의사결정을 할 수 있고 더 나아가 건강한 금융 가치관을 형성할 수 있다.

2) 금융 상품 이용 활성화

낯설었던 금융 용어와 개념을 이해하게 되면, 자연스럽게 다양한 금융 상품에 대한 관심이 높아지며, 이를 실제 금융 상품 가입으로도 연결할 수 있다.

예를 들어, 많은 사용자들이 금융 상품에 접근하는 데 어려움을 겪는 주요 원인은 복잡한 용어와 개념에 대한 이해 부족이다. 본 아이디어는 이러한 장벽을 낮추어 사용자들이 보다 쉽게 금융 시장에 진입하여 금융 상품을 이용할 수 있도록 도와준다.

3) 사용자 편의성 증진 및 긍정적인 기업 이미지 제고

사용자가 자신의 수준에 맞는 맞춤형 설명을 제공받음으로써 금융 지식 및 금융 상품에 대해 쉽게 이해하고, 의사결정을 내릴 수 있게 되는 것은 사용자의 편의성을 크게 향상시킨다. 만약 금융 기업에서 해당 서비스를 제공한다면, 해당 기업에 대한 긍정적인 이미지를 형성하고, 고객 충성도를 강화하는 데 기여할 수 있다. 또한, 이러한 사용자 중심의 접근 방식은 차별화된 경쟁력을 제공하여, 더욱 많은 고객을 플랫폼으로 유치할 수 있을 것으로 기대한다.

4) 금융 용어 미숙으로 인한 금융 사고 예방

금융 용어에 대한 이해 부족은 잘못된 의사결정을 초래할 수 있으며, 이는 장기적으로 금융 사고나 손실로 이어질 가능성이 있다. 본 서비스는 사용자가 명확한 금융 용어 이해

를 바탕으로 의사결정 결정을 내리도록 도와 금융 사고를 예방할 수 있다. 특히, 금융 지식이 낮은 사용자들이 금융 경험이 부족한 상태에서 발생할 수 있는 위험을 줄이는 데 기여할 수 있다.

9. PoC 설계 개념도 및 주요 코드 설명

본 기획서에 대한 PoC 구현은 사용자의 질문으로 금융 지식 수준을 측정하는 기능 구현을 목표로 한다. 또한 RAG 답변 성능 향상을 부차적인 목표로 진행한다.

9.1 프로그램 설계 개념도

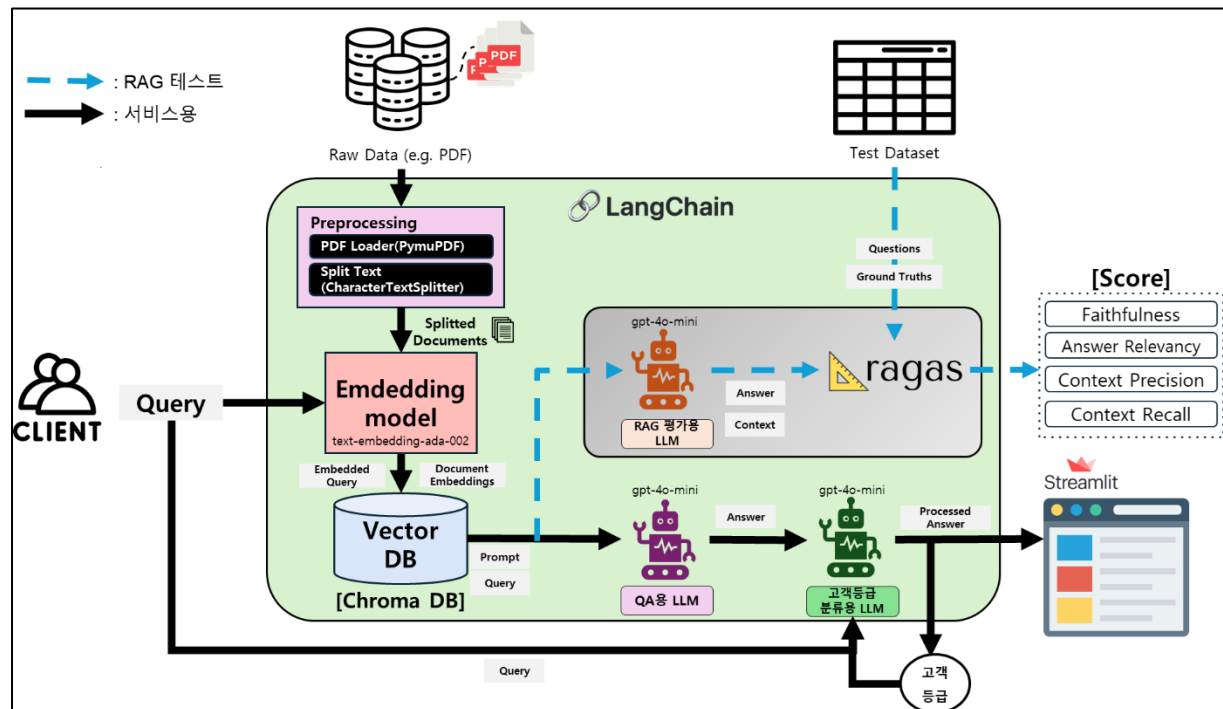


그림 10. PoC 프로그램 설계 개념도

9.2 구현 프로젝트 구조

fsi_idea_poc

```
|
|—— /data      # RAG에 필요한 PDF와 Question & Ground-Truth 세트가 저장된 폴더
|—— app.py      # Streamlit 인터페이스 코드
|—— chatbot.py  # PDFRAGChatbot 클래스 및 관련 로직
|—— eval_rag.py # RAGAS를 이용한 RAG 평가 클래스 EVALRAGBot 관련 로직
|—— customer_classification.py # CustomerClassifier 클래스 및 관련 로직
|—— config.py  # 환경 변수 관리 및 설정
|—— utils.py   # 유틸리티 함수들 (예: 파일 처리)
|—— .env       # 환경 변수 설정 파일
```

추가적으로 VectorDB를 ChromaDB를 활용해 구현함으로써 루트 폴더(디렉토리)에 /chroma_db 폴더가 생성된다.

이외에도 streamlit을 이용한 채팅에 사용되었던 정보를 저장하는 chat_log.csv와 RAG 평가에 이용한 데이터셋 CSV 파일, RAGAS를 이용한 평가 전반에 대한 결과값이 CSV 파일로 루트 폴더에 생성된다.

9.3 구현 파일 개요

파일	기능
app.py*	Streamlit Interface를 구현한 파일
chatbot.py	PDF 파일 전처리 및 embedding model을 활용한 vector DB 구축, RAG chain 생성 등 전반적인 RAG가 적용된 PDFRAGChatbot 클래스가 내장된 파일
eval_rag.py*	오픈소스 Python 라이브러리인 RAGAS를 이용해 RAG의 검색 및 생성 성능을 평가하는 클래스(EVALRAGBot)가 내장된 파일
customer_classification.py	사용자의 질문에 대해 사용자의 투자 수준을 평가하고, RAG에서 제공한 답변 가공을 수행하는 클래스(CustomerClassifier)가 구현된 코드
config.py	Streamlit 서비스 구동을 위한 환경 변수 관리 및 설정, 프로젝트 폴더(디렉토리)를 설정하고 RAG 및 답변 가공을 위한 서비스의 디버깅을 위해 Langsmith 이용을 위한 환경 변수 설정을 하는 파일
utils.py	파일 처리와 같은 유틸리티 함수들이 내장된 파일
.env	환경 변수 설정 파일 OpenAI의 API키와 Langsmith의 API키 저장

* streamlit 서비스는 streamlit run app.py 명령어로 실행

* RAGAS를 이용한 rag 성능 평가는 python eval_rag.py 명령어로 실행하여 결과 도출

9.4 단계별 프로그램 주요 코드 캡처/설명 및 결과화면

1) utils.py - 주요 함수 설명

```
def load_ground_truth(df):
    ground_truth = {}

    # DataFrame의 각 행이 하나의 ETF에 해당한다고 가정
    for idx, row in df.iterrows():
        etf_name = row['상품명']
        ground_truth[etf_name] = {
            '투자등급': row['답변1 (투자등급)'],
            '기초지수': row['답변2 (기초지수)'],
            '투자포인트': row['답변3 (투자 포인트)'],
            '주요보유현황': row['답변4 (주요보유현황)'],
            '운용성과': row['답변5 (운용성과)']
        }
    return ground_truth
```

"pdf_목록.xlsx"에 저장된 상품명과 각 상품에 대해 설정한 Ground-Truth값을 파일에서 추출하는 함수이다.

```
def prepare_questions_and_gts(ground_truth):
    questions = []
    gts = []

    for etf_name, gt_data in ground_truth.items():
        # ETF 종목명을 질문에 포함
        questions.append(f"{etf_name}의 투자등급은 몇 등급입니까?")
        questions.append(f"{etf_name}의 기초 지수는 어떻게 되나요?")
        questions.append(f"{etf_name}의 투자 포인트는 무엇인가요?")
        questions.append(f"{etf_name}의 주요 종목 현황은 어떻게 되나요?")
        questions.append(f"{etf_name}의 운용 성과는 어떻게 되나요?")

        # 해당하는 ground truth를 답변으로 추가
        gts.append(gt_data['투자등급'])
        gts.append(gt_data['기초지수'])
        gts.append(gt_data['투자포인트'])
        gts.append(gt_data['주요보유현황'])
        gts.append(gt_data['운용성과'])

    return questions, gts
```

load_ground_truth 함수를 통해 추출한 Ground-Truth를 가공하여 Question-Gt 쌍을 생성하도록 하는 함수이며, RAG 성능 평가 시 사용될 함수이다.

2) chatbot.py - 주요 함수 설명

```
class PDFRAGChatbot:
    def __init__(self, prj_dir, openai_api_key):
        self.project_dir = prj_dir
        self.openai_api_key = openai_api_key
        self.embeddings_model = OpenAIEmbeddings(model="text-embedding-ada-002", openai_api_key=self.openai_api_key)
        self.vectorstore = None # Chroma 벡터 스토어 초기화
        self.retriever = None # retriever 초기화
        self.rag_chain = None # RAG 체인 초기화
```

RAG 구현을 위해 설계한 PDFRAGChatbot 클래스이다. 생성자를 통해, prj_dir(프로젝트 루트 디렉토리)와 openai_api_key를 받아와 클래스의 속성으로 사용하며, RAG에서 사용할 embedding_model을 OpenAI의 text-embedding-ada-002 모델로 초기화한다. vectorstore는 ChromaDB를 위해 사용할 변수이며, retriever는 문서 검색 기능을 위해 설정한 변수이다. 또한, LangChain의 LCEL에 기반한 파이프라인 구축을 위해 qa_chain을 생성한다.

```
def split_text(self, documents):
    try:
        text_splitter = CharacterTextSplitter(
            separator=r"[.!?]\s+",
            chunk_size=1500,
            chunk_overlap=300,
            length_function=len,
            is_separator_regex=True,
        )
        return text_splitter.split_documents(documents)
    except Exception as e:
        print(f"텍스트 분할 중 오류 발생: {str(e)}")
        return []
```

PDFRAGChatbot 클래스에서 구현한 split_text 메소드이다. LangChain에서 제공하는 CharacterTextSplitter를 이용해 text_splitter를 설정한다. 이 때, 정규표현식으로 표현된 구분자(separator)를 기준으로 1500자의 텍스트로 chunk를 생성하며, 각 chunk간의 겹치는 텍스트(chunk_overlap)는 300자로 설정한다. 이 함수는 Streamlit 상에 업로드된 PDF 파일들을 청크 단위로 분할하기 위해 사용된다.

```
def create_vectorstore(self, docs):
    try:
        self.vectorstore = Chroma.from_documents(
            documents=docs,
            embedding=self.embeddings_model,
            collection_name="collection_legal_pdf",
            collection_metadata={'hnsw:space': 'ip'},
            persist_directory=os.path.join(self.project_dir, "chroma_db")
        )
        self.retriever = self.vectorstore.as_retriever(search_kwargs={"k": 3})
        print("벡터 스토어가 성공적으로 생성되었습니다.")
    except Exception as e:
        print(f"벡터 스토어 생성 중 오류 발생: {str(e)}")
```

PDFRAGChatbot 클래스에서 구현한 create_vectorstore 메소드이다. ChromaDB를 활용해 vectorstore에 vectorDB를 저장한다. 이 때, chunk 단위로 분할된 PDF 파일을 documents로, embedding_model은 OpenAI의 text-embedding-ada-002를 활용한다.

벡터 임베딩, 문서, 메타데이터 등을 저장하는 공간을 collection이라고 한다. 해당 공간의 이름을 'collection_legal_pdf'로 설정하였으며, hnsw:space를 통해 유사도 메트릭을 설정하는데, 이 때 HNSW의 알고리즘 검색 공간을 ip(inner product, 내적)을 기준으로 설정한다.

벡터 스토어 생성이 완료된 후, retriever는 as_retriever 함수로 설정한다. retriever는 생성한 vectorstore 기반으로 가장 관련 있는 문서 3개를 탐색하도록 설정한다.

```
def setup_rag_chain(self):
    try:
        template = '''다음 문맥을 기반으로 질문에 답변하세요:
        {context}

        질문: {query}
        '''
        prompt = ChatPromptTemplate.from_template(template)
        llm = ChatOpenAI(model="gpt-4o-mini", temperature=0, max_tokens=1000, openai_api_key=self.openai_api_key)
        format_docs = lambda docs: "\n\n".join([d.page_content for d in docs])

        self.rag_chain = (
            {"context": self.retriever | format_docs, "query": RunnablePassthrough()}
            | prompt
            | llm
            | StrOutputParser()
        )

        print("RAG 체인이 성공적으로 설정되었습니다.")

    except Exception as e:
        print(f"RAG 체인 설정 중 오류 발생: {str(e)}")
```

PDFRAGChatbot 클래스에서 구현한 setup_rag_chain 메소드이다. ChatPromptTemplate.from_template 메소드를 이용해 template 바탕의 prompt 객체를 생성한다. llm은 OpenAI에서 제공하는 gpt-4o-mini를 사용하였으며, 답변의 일관성을 부여하기 위해 temperature를 0으로 설정하였고, 생성되는 답변의 최대 토큰 수를 1000으로 설정하였다. 이 때, llm에는 AIMessage 객체로 답변이 저장된다. format_docs에 검색된 문서 리스트에서 문서 내용을 가져와 하나의 텍스트로 결합하는 람다 함수를 정의한다.

LCEL 기반으로 RAG 파이프라인을 구성하여 rag_chain을 생성하였다. retriever가 검색한 문서를 벡터 스토어에서 찾아 context 키와 결합된 value 값에 저장하며, 이 때 format_docs에서 정의한 함수를 활용한다. query는 해당 rag_chain을 invoke할 때 인자로 넘어가며, RunnablePassthrough를 통해 그대로 다음 단계인 prompt에 전달된다. 생성한 프롬프트 템플릿을 사용해 검색된 문서 및 질문을 프롬프트로 변환한다. 생성된 프롬프트를 gpt-4o-mini에 전달하여 답변을 생성한 뒤, StrOutputParser를 통해 문자열로 파싱되어 최종

결과를 도출할 수 있는 구조로 rag 파이프라인을 설계한다.

```
def generate_response(self, query):
    if self.rag_chain:
        try:
            result = self.rag_chain.invoke(query)
            return result
        except Exception as e:
            print(f"답변 생성 중 오류 발생: {str(e)}")
            return "답변 생성 중 오류가 발생했습니다."
    else:
        return "RAG 체인이 설정되지 않았습니다."
```

PDFRAGChatbot 클래스에서 구현한 generate_response 메소드이다. query를 invoke 함수를 통해 앞서 설정한 rag_chain을 호출해주면 result에 RAG를 통해 생성된 답변을 얻을 수 있다.

3) customer_classification.py – 주요 함수 설명

```
class CustomerClassifier:
    def __init__(self, model_name="gpt-4o-mini"):
        # LangChain LLM 초기화
        self.llm = ChatOpenAI(model_name=model_name, temperature=0.8)

        # Few-shot prompting을 위한 예시 추가
        few_shot_examples = """
        [Newcomer]
        - 질문: ETF가 무엇인가요?
        -> 원래 답변 : ETF는 특정지수 또는 가격의 수익률을 추종하는 인덱스
            미국인 일반은 ETF를 주식보다 쉽게 이해할 수 있는 상품으로 본다.

        # PromptTemplate 설정 (Few-shot examples 포함)
        self.prompt_template = PromptTemplate(
            input_variables=["query", "rag_answer", "conversation_history", "level_percentage"],
            template=f"""
            다음은 고객의 질문과 그에 따른 답변입니다.

            당신은 아래 3가지 단계를 거칩니다.
            1. 당신은 고객의 질문을 기반으로 고객의 수준(Newcomer/Learner/Investigator)을 분류합니다.
            [Newcomer]: ETF와 같은 기초적인 개념을 묻는 유형으로, 기본적인 용어, 개념 및 특징에 대한 질문
            [Learner]: ETF의 여러 종류 및 파생상품에 대한 이해가 부족한 고객을 대상으로, 기본 용어 지식
            [Investigator]: 구체적이고 정밀한 정보를 요구하는 유형으로, 세금, 환율 위험, 구체적 수치
            2. 현재까지의 고객 수준에 맞춰 적절하게 RAG 답변을 가공합니다.
            3. 최종적으로 질문에 대한 가공된 답변과 고객의 수준을 도출하세요.

            고객 수준별 가공된 답변 예시는 다음과 같습니다.
            {few_shot_examples}

            대화 히스토리:
            {{conversation_history}}

            현재 고객의 질문을 통해, 고객의 수준을 다음 중 하나로 분류하세요: [Newcomer, Learner, Inve
            고객의 수준:

            질문: {{query}}
            RAG 답변: {{rag_answer}}

            이전까지 고객의 수준 비율은 다음과 같습니다: {{level_percentage}}
            현재 고객의 수준과 이전까지 고객의 수준의 상태를 반영해서, 고객 질문에 대한 고객의 수준에 맞
            가공된 답변:
            """
        )

    # LLMChain 생성
    self.chain = self.prompt_template | self.llm
    self.levels_count = {"Newcomer": 0, "Learner": 0, "Investigator": 0}
```

사용자 질문(query)에 따른 투자 수준 분류와 사용자의 투자 수준에 따라 RAG로 도출한 AI의 답변을 가공하기 위해 설계한 CustomerClassifier 클래스이다.

default llm 모델은 gpt-4o-mini로 설정하였으며, 가공된 답변의 다양성을 주기 위해 temperature를 0.8로 설정하였다. 프롬프트 템플릿에 few-shot-prompting 기법을 적용하였다. few-shot-prompting이란 프롬프트 엔지니어링 기법 중 하나로, 몇 가지 예시를 제공하여 llm 모델이 프롬프트를 바탕으로 보다 나은 성능(보

다 나은 답변)을 발휘할 수 있도록 만드는 방법이다. llm 모델이 처리해야 하는 작업이 복합적이기 때문에 상세하게 프롬프트를 작성하여 의도한 답변(customer level을 평가하고, conversation history와 customer level percentage를 반영하여 답변을 가공)을 생성할 수 있도록 한다.

LCEL에 기반해 LangChain 파이프라인을 설정해주었고, level_count 딕셔너리를 초기화하여 사용자의 질문에 대해 평가한 사용자의 투자 수준을 카운트하여 기록한다.

```
def classify(self, query, rag_answer, chain):
    # 이전 대화 히스토리 생성
    conversation_history = "\n".join(
        [f"질문: {q['질문']}\n답변: {q['가공된 답변']}" for q in chain]
    )

    # 고객 수준 비율 계산
    total = sum(self.levels_count.values())
    if total > 0:
        level_percentage = {
            level: f"({count / total} * 100:.1f)%" for level, count in self.levels_count.items()
        }
    else:
        level_percentage = {
            "Newcomer": "0%", "Learner": "0%", "Investigator": "0%"
        }

    # LLM이 결합된 Chain을 이용해 결과 생성
    result = self.chain.invoke({
        "query": query,
        "rag_answer": rag_answer,
        "conversation_history": conversation_history,
        "level_percentage": level_percentage
    })

    # result에서 텍스트를 추출하기 위한 코드
    if hasattr(result, 'content'):
        if isinstance(result.content, str):

# LLMChain 결과에서 가공된 답변과 고객 수준 추출
refined_answer, customer_level = self.extract_response(result_text)

if customer_level in self.levels_count:
    self.levels_count[customer_level] += 1

return {
    "refined_answer": refined_answer,
    "customer_level": customer_level
}
```

CustomerClassification에서 구현한 classify 메소드이다. User의 query(질문)과 answer(RAG를 통해 생성한 답변) 그리고 streamlit 세션에 저장된 conversation_history를 인자로 받아와 이전 대화 히스토리를 리스트로 생성하고, 현재 사용자의 투자 수준(Newcomer/Learner/Investigator)에 대한 비율을 계산한다.

query(질문), rag답변, 이전 대화 내역, level_percentage를 바탕으로 앞서 생성했던 chain을 호출하여 그 결과를 result에 저장한다.

저장된 result는 string 형태로 가공되어 result_text에 저장되며 extract_response 함수를 이용해 refined_answer와 customer_level을 추출한다. customer_level을 바탕으로 level_count를 1회 증가시키고, 추출한 refined_answer와 customer_level은 딕셔너리 형태로 반환하여 Streamlit에서 사용할 수 있도록 한다.

```
def determine_final_level(self):
    return max(self.levels_count, key=self.levels_count.get)
```

CustomerClassification에서 구현한 determine_final_level 메소드이다. config.py에서 설정한 질문 횟수에 도달하면 해당 메소드를 실행하도록 app.py에 구현하였으며, levels_count 딕셔너리에서 가장 큰 value를 가지는 key값을 return한다.

4) app.py – 주요 함수 설명


```
def initialize_chatbot():
    if 'chatbot' not in st.session_state:
        st.session_state.chatbot = PDFRAGChatbot(PRJ_DIR, OPENAI_API_KEY)

def initialize_classifier():
    if 'classifier' not in st.session_state:
        st.session_state.classifier = CustomerClassifier(model_name="gpt-4o-mini")

def initialize_conversation():
    if 'conversation_history' not in st.session_state:
        st.session_state.conversation_history = []
    if 'qa_round' not in st.session_state:
        st.session_state.qa_round = 0
```

위 세 가지 함수는 Streamlit이 구동됨과 동시에 앞서 클래스로 설계했던 chatbot 인스턴스, classifier 인스턴스와 동시에 conversation_history 리스트, qa_round 변수를 사용자의 session에 저장하도록 하는 함수이다.

```
def handle_pdf_upload(uploaded_files):
    if uploaded_files:
        all_docs = []
        for uploaded_file in uploaded_files:
            file_path = save_uploaded_file(uploaded_file)
            data = st.session_state.chatbot.load_pdf(file_path)
            docs = st.session_state.chatbot.split_text(data)
            all_docs.extend(docs)

        if 'vectorstore_created' not in st.session_state:
            st.session_state.chatbot.create_vectorstore(all_docs)
            st.session_state.chatbot.setup_rag_chain()
            st.session_state.vectorstore_created = True

    st.write(f"모든 PDF 파일이 성공적으로 처리되었습니다. 총 {len(all_docs)} 페이지가 있습니다.")
```

utils.py에 작성했던 save_uploaded_file 함수를 통해 업로드한 다수의 pdf 파일을 저장해 경로를 추출한다. vectorstore가 생성되지 않았으면, vectorstore를 생성하고 rag_chain을 만든다.

```
def main():
    st.title('FSI Chatbot by. water_fight')

    initialize_chatbot()
    initialize_classifier()
    initialize_conversation()

    st.subheader('PDF 파일 업로드 (여러 개 선택 가능)')
    uploaded_files = st.file_uploader("PDF 파일을 업로드하세요.", type="pdf", accept_multiple_files=True)

    if uploaded_files:
        handle_pdf_upload(uploaded_files)

    if st.session_state.qa_round >= QUESTION_LIMIT:
        st.write(f"{QUESTION_LIMIT}회의 질문과 답변에 따른 고객님의 최종등급은 {st.session_state.classifier.determine_final_level()}입니다.")
    else:
        if "messages" not in st.session_state:
            st.session_state.messages = [{"role": "assistant", "content": "투자설명서를 읽고 궁금한 점을 질문하세요!"}]

        for msg in st.session_state.messages:
            st.chat_message(msg["role"]).write(msg["content"])

        if query := st.chat_input("질문을 입력하세요."):
            st.session_state.messages.append({"role": "user", "content": query})
            st.chat_message("user").write(query)
```

```

# Fetch the response and classify customer
response = st.session_state.chatbot.generate_response(query)

classification_result = st.session_state.classifier.classify(query, response, st.session_state.conversation_history)
refined_answer = classification_result["refined_answer"]
customer_level = classification_result["customer_level"]

print(f"질문: {query}\nRAG 답변: {response}\n가공된 답변: {refined_answer}\n해당 질문에 대한 고객의 투자등급: {customer_level}\n")

# Update conversation history
st.session_state.conversation_history.append({
    '질문': query,
    '가공된 답변': refined_answer,
    '고객 등급': customer_level
})

combined_response = f"위 질문으로 판단한 고객 투자 수준: {customer_level}\n\n답변: {refined_answer}"

st.session_state.messages.append({"role": "assistant", "content": combined_response})
st.chat_message("assistant").write(combined_response)

# Chat logging at CSV
save_log_to_csv('./chat_log.csv', query, response, refined_answer, customer_level, st.session_state.classifier.levels_count)

st.session_state.qa_round += 1

if __name__ == "__main__":
    main()

```

앞선 3개의 초기화 함수를 실행함으로써 인스턴스를 생성 및 세션에 저장하고, handle_pdf_upload 함수를 통해 업로드한 PDF 파일들을 처리한다.

config.py에서 정의한 QUESTION_LIMIT에 도달하면, 최종등급을 산출하고, 아닌 경우엔 챗봇 서비스가 지속적으로 실행된다.

session_state에 messages라는 키가 없으면 기본값을 설정하도록 한다. 기본값은 챗봇(assistant)의 환영 메시지이다. messages의 모든 메시지를 반복문을 돌며, 각 메시지의 역할(role)과 내용(content)을 읽어 Streamlit의 chat_message 메소드를 이용해 대화창에 출력한다.

chat_input 메소드를 활용해 사용자의 질문을 입력 받다. 이 때의 질문은 query라는 변수에 저장되고, 현재 session_state에 user 역할로 메시지가 추가되며, 대화창에 해당 질문이 출력된다.

앞서 chatbot.py의 PDFRAGChatbot에 정의한 generate_response 메소드를 활용해 RAG 답변을 생성하고, query(사용자의 현재 질문), response(이에 대한 RAG 답변), session_state에 저장된 대화 내역을 바탕으로 CustomerClassification.py에서 설계한 CumstomerClassifier 클래스에 정의된 classify 메소드를 실행하여 가공된 답변(refined_answer)과 사용자 투자 수준(customer_level)을 받아온다.

위 결과를 conversation_history 리스트에 저장하고, 챗봇(assistant)가 출력하기 위한 combined_response(가공된 응답과 사용자의 투자 등급을 통합한 하나의 응답 메시지)를 만듭니다. combined_response는 대화창에 챗봇(assistant)의 메시지로 추가된다.

utils.py에 작성한 save_log_to_csv 함수를 이용해 사용자의 질문, RAG 응답, 가공된 답변, 사용자의 투자 등급, 사용자의 투자 등급 카운트한 levels_count를 CSV 파일에 저장한다.

대화 세션에서 몇 번째 질문인지 추적하기 위해 qa_round값을 증가시킨다.

[결과 화면 캡처]



FSI Chatbot by. water_fight

PDF 파일 업로드 (여러 개 선택 가능)

PDF 파일을 업로드하세요.



Drag and drop files here

Limit 200MB per file • PDF

Browse files



월간 (20).pdf 269.4KB



월간 (19).pdf 270.2KB



월간 (18).pdf 271.6KB



Showing page 1 of 14



모든 PDF 파일이 성공적으로 처리되었습니다. 총 314 페이지가 있습니다.



투자설명서를 읽고 궁금한 점을 질문하세요!



ETF가 뭐야?



위 질문으로 판단한 고객의 금융 지식 수준: Newcomer

답변: ETF는 "상장지수펀드"의 약자로, 주식시장에 상장되어 거래되는 펀드입니다. 쉽게 말해, ETF를 사면 주식처럼 사고팔 수 있는 투자 수단을 가지게 됩니다. 이 펀드는 특정 지수나 자산군(예: 코스피 지수 또는 특정 산업군의 주식)을 따라가도록 설계되어 있습니다. 즉, ETF는 그 지수의 성과를 본따서 투자자에게 수익을 제공합니다. 또한, ETF는 어떤 투자 자산을 포함하고 있는지, 어떻게 배당금을 지급하는지에 대한 정보를 담고 있는 투자 설명서를 반드시 읽어보는 것이 중요합니다. 투자할 때는 손실이 발생할 수 있으니 주의해야 합니다.

질문을 입력하세요.



TIGER MKF배당귀족 ETF의 기초지수에 대해서 설명해줘.



위 질문으로 판단한 고객의 금융 지식 수준: Investigator

답변: TIGER MKF배당귀족 ETF의 기초지수는 MKF 배당귀족지수로, 매일경제신문과 FnGuide가 공동 개발한 지수입니다. 이 지수는 10년 연속으로 주당 배당금이 증가했거나 유지된 기업들 중에서 배당 수익률이 높은 기업에 투자하는 지수입니다. 따라서, 안정적인 배당금을 지급하는 기업에 중점을 두고 있으며, 이러한 기업들에 대한 투자를 통해 배당 수익을 추구하는 투자 전략을 가지고 있습니다. 구체적으로, 이 지수에 포함되는 종목들은 배당 안정성과 수익성을 동시에 고려하여 선정되며, ETF 투자자는 이를 통해 안정적인 배당 수익을 기대할 수 있습니다.

질문을 입력하세요.





TIGER 게임TOP10 ETF의 주요 구성종목에 대하여 설명해줘.



위 질문으로 판단한 고객의 금융 지식 수준: Investigator

답변: TIGER 게임TOP10 ETF의 주요 구성종목은 다음과 같습니다:

1. **크래프톤 (32.52%)**: '배틀그라운드'와 같은 인기 게임을 개발 및 배급하는 회사입니다.
2. **넷마블 (23.47%)**: '리니지2 레볼루션', '세븐나이츠' 등 다양한 모바일 게임을 개발하는 회사입니다.
3. **엔씨소프트 (22.28%)**: MMORPG 분야에서 강력한 입지를 가진 회사로, '리니지' 시리즈와 '블레이드 & 소울' 등을 개발했습니다.
4. **펄어비스 (5.78%)**: '검은사막'으로 유명한 게임 개발사로, 뛰어난 그래픽과 게임성으로 높은 평가를 받고 있습니다.
5. **카카오게임즈 (3.69%)**: 다양한 모바일 및 PC 게임을 서비스하는 플랫폼입니다.
6. **위메이드 (3.28%)**: '미르의 전설' 시리즈로 잘 알려진 MMORPG 및 모바일 게임 개발사입니다.
7. **더블유게임즈 (2.64%)**: 슬롯 및 카지노 게임을 개발하는 회사로 모바일 게임 시장에서 활동하고 있습니다.
8. **넥슨게임즈 (2.24%)**: 넥슨의 자회사로 다양한 장르의 게임을 개발하고 있습니다.
9. **컴투스 (1.67%)**: '서머너즈 워'와 같은 인기 모바일 게임을 보유하고 있는 회사입니다.
10. **NHN (1.54%)**: 게임 및 IT 서비스 기업으로 여러 게임과 플랫폼 서비스를 제공합니다.

이 ETF는 대한민국 게임 산업의 대표 기업 10개에 투자하여, 게임 산업의 성장 가능성을 반영하는 것을 목표로 하고 있습니다. 각 회사의 비중이 ETF 성과에 미치는 영향을 고려하며, 이러한 구성 종목들은 게임 산업의 트렌드와 변동성에 따라 성과가 달라질 수 있습니다.

질문을 입력하세요.





TIGER 글로벌AI액티브 ETF에 투자했을 때의 세금 계산 방식에 대하여 설명해줘.



위 질문으로 판단한 고객의 금융 지식 수준: Investigator

답변: TIGER 글로벌AI액티브 ETF에 투자했을 때의 세금 계산 방식은 다음과 같습니다:

1. **과세 원칙:** ETF는 집합투자기구로 분류되어 기본적으로 소득세가 부과되지 않습니다. 즉, 투자자가 ETF의 수익을 실현하기 전까지는 별도의 세금 부담이 없습니다.
2. **원천징수 세율:** 거주자 및 일반법인이 집합투자기구를 통해 발생한 과세 이익에 대해서는 15.4%의 세율(지방소득세 포함)로 원천징수됩니다. 이는 배당 소득에 적용됩니다.
3. **종합과세:** 연간 금융소득의 합계가 기준금액(현재 2000만원)을 초과할 경우, 기준금액을 초과하는 금액은 다른 종합소득과 합산되어 개인소득세율에 따라 종합과세됩니다. 이 경우, 세율은 누진세율로 적용됩니다.
4. **보유기간 과세:** 2010년 7월 1일부터, 상장지수 집합투자기구의 경우 보유기간 중 발생한 이익에 대해 배당소득세가 과세됩니다. 즉, ETF의 배당금 수익은 해당 시점에서 세금을 부과 받게 됩니다.
5. **특별한 경우:** 다만, 국내주식형 ETF는 보유기간 과세의 적용을 받지 않으며, 거래 시 발생하는 세금은 국내주식 직접투자자와 비교할 때 상당히 유리할 수 있습니다.

투자자는 자신의 금융소득 상황을 고려하여 세금 부담을 미리 계획하는 것이 중요합니다. 각 ETF의 세금 처리 방식은 다를 수 있으므로, 이를 충분히 이해하고 투자 결정을 내리는 것이 필요합니다.

질문을 입력하세요.



TIGER 미국30년국채프리미엄액티브(H) ETF의 리밸런싱이 있는지, 리밸런싱이 있다면 주기는 언제인지 설명해줘.



위 질문으로 판단한 고객의 금융 지식 수준: Investigator

답변: TIGER 미국30년국채프리미엄액티브(H) ETF의 리밸런싱은 KEDI US Treasury 30Y Weekly Covered Call 30 Index를 기초로 하여 이루어집니다. 이 지수는 매주 리밸런싱을 실시하며, ETF의 포트폴리오도 이에 맞춰 조정됩니다. 따라서, TIGER 미국30년국채프리미엄액티브(H) ETF는 매주 새로운 구성 비율에 따라 리밸런싱이 이루어지므로, 투자자는 이 점을 고려하여 투자 결정을 내릴 필요가 있습니다. 더 구체적인 리밸런싱 일정이나 방법은 해당 ETF의 공식 웹사이트나 투자 설명서를 통해 확인할 수 있습니다.

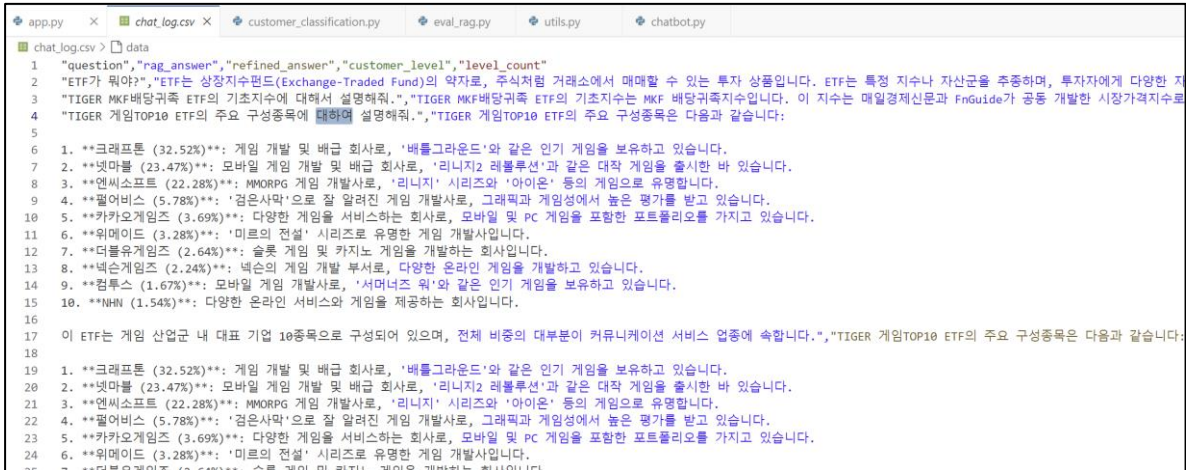
질문을 입력하세요.



모든 PDF 파일이 성공적으로 처리되었습니다. 총 206 페이지가 있습니다.

5회의 질문과 답변에 따른 고객의 금융 지식 수준은 Investigator입니다.

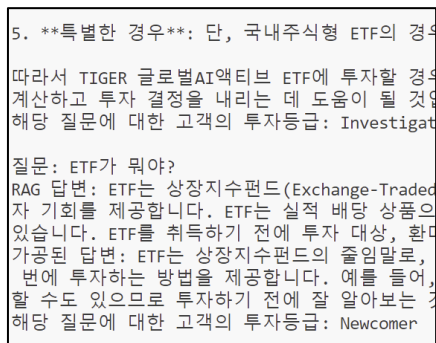
아래 그림과 같이 chat_log.csv를 통해 질문 및 rag답변 등의 이력을 확인할 수 있다.



```
1 "question","rag_answer","refined_answer","customer_level","level_count"
2 "ETF가 뭐야?","ETF는 상장지수펀드(Exchange-Traded Fund)의 약자로, 주식처럼 거래소에서 매매할 수 있는 투자 상품입니다. ETF는 특정 지수나 자산군을 추종하며, 투자자에게 다양한 지
3 "TIGER MKF배당귀족 ETF의 기초지수에 대해서 설명해줘","TIGER MKF배당귀족 ETF의 기초지수는 MKF 배당귀족지수입니다. 이 지수는 매일경제신문과 FnGuide가 공동 개발한 시장가격지수로
4 "TIGER 게임TOP10 ETF의 주요 구성종목에 대하여 설명해줘","TIGER 게임TOP10 ETF의 주요 구성종목은 다음과 같습니다:
5
6 1. **크래프톤 (32.52%):** 게임 개발 및 배급 회사로, '배틀그라운드'와 같은 인기 게임을 보유하고 있습니다.
7 2. **넷마블 (23.47%):** 모바일 게임 개발 및 배급 회사로, '리니지2 레볼루션'과 같은 대작 게임을 출시한 바 있습니다.
8 3. **엔씨소프트 (22.28%):** MMORPG 게임 개발사로, '리니지' 시리즈와 '아이온' 등의 게임으로 유명합니다.
9 4. **펄비스 (5.78%):** '검은사막'으로 잘 알려진 게임 개발사로, 그래픽과 게임성에서 높은 평가를 받고 있습니다.
10 5. **카카오게임즈 (3.69%):** 다양한 게임을 서비스하는 회사로, 모바일 및 PC 게임을 포함한 포트폴리오를 가지고 있습니다.
11 6. **위메이드 (3.28%):** '미르의 전설' 시리즈로 유명한 게임 개발사입니다.
12 7. **더블유게임즈 (2.64%):** 슬롯 게임 및 카지노 게임을 개발하는 회사입니다.
13 8. **넥슨게임즈 (2.24%):** 넥슨의 게임 개발 부서로, 다양한 온라인 게임을 개발하고 있습니다.
14 9. **컴투스 (1.67%):** 모바일 게임 개발사로, '서머너즈 워'와 같은 인기 게임을 보유하고 있습니다.
15 10. **NHN (1.54%):** 다양한 온라인 서비스와 게임을 제공하는 회사입니다.
16
17 이 ETF는 게임 산업군 내 대표 기업 10종목으로 구성되어 있으며, 전체 비중의 대부분이 커뮤니케이션 서비스 업종에 속합니다. ", "TIGER 게임TOP10 ETF의 주요 구성종목은 다음과 같습니다:
18
19 1. **크래프톤 (32.52%):** 게임 개발 및 배급 회사로, '배틀그라운드'와 같은 인기 게임을 보유하고 있습니다.
20 2. **넷마블 (23.47%):** 모바일 게임 개발 및 배급 회사로, '리니지2 레볼루션'과 같은 대작 게임을 출시한 바 있습니다.
21 3. **엔씨소프트 (22.28%):** MMORPG 게임 개발사로, '리니지' 시리즈와 '아이온' 등의 게임으로 유명합니다.
22 4. **펄비스 (5.78%):** '검은사막'으로 잘 알려진 게임 개발사로, 그래픽과 게임성에서 높은 평가를 받고 있습니다.
23 5. **카카오게임즈 (3.69%):** 다양한 게임을 서비스하는 회사로, 모바일 및 PC 게임을 포함한 포트폴리오를 가지고 있습니다.
24 6. **위메이드 (3.28%):** '미르의 전설' 시리즈로 유명한 게임 개발사입니다.
25 7. **더블유게임즈 (2.64%):** 슬롯 게임 및 카지노 게임을 개발하는 회사입니다.
```

그림 11. chat_log.csv의 화면 캡처

terminal의 동일한 결과를 print하여 terminal 창에서도 이를 확인할 수 있다.



```
5. **특별한 경우:** 단, 국내주식형 ETF의 경우
따라서 TIGER 글로벌AI액티브 ETF에 투자할 경우
계산하고 투자 결정을 내리는 데 도움이 될 것
해당 질문에 대한 고객의 투자등급: Investigator

질문: ETF가 뭐야?
RAG 답변: ETF는 상장지수펀드(Exchange-Traded
자 기회를 제공합니다. ETF는 실제 배당 상품
있습니다. ETF를 취득하기 전에 투자 대상, 환
가공된 답변: ETF는 상장지수펀드의 줄임말로,
번에 투자하는 방법을 제공합니다. 예를 들어,
할 수도 있으므로 투자하기 전에 잘 알아보는
해당 질문에 대한 고객의 투자등급: Newcomer
```

그림 12. terminal 창에서의 화면 캡처

5) eval_rag.py - 주요 함수 설명

```

class EvalRAGBot(PDFRAGChatbot):
    def __init__(self, pdf_files, questions, gts, openai_api_key, prj_dir):
        super().__init__(prj_dir, openai_api_key)
        self.pdf_files = pdf_files
        self.questions = questions
        self.gts = gts

        self.answers = []
        self.contexts = []

        # 문서 및 파이프라인 초기화
        self.vectorstore = None
        self.qa_chain = None
        self.embeddings = self.embeddings = OpenAIEmbeddings(
            model="text-embedding-ada-002",
            openai_api_key=self.openai_api_key)
        self.llm = None

        # 초기화 시 모든 과정을 처리
        self.setup_rag_pipeline()

```

chatbot.py에 설계한 PDFRAGChatbot 클래스를 상속하여 EvalRAGBot 클래스를 설계한다. PDFRAGChatbot의 project_dir와 openai_api_key를 가져와 prj_dir와 openai_api_key를 저장하고, pdf_files에는 PDF 파일 경로 리스트를 questions에는 평가에 사용할 질문 리스트, gts에는 각 질문에 대한 정답(ground-truth) 리스트를 저장한다. answers와 contexts에는 RAGS 평가에 필요한 각 질문과 ground_truth에 대하여 RAG가 생성한 답변, 이용한 contexts가 리스트 형식으로 저장될 예정이므로 빈 리스트를 만들어 놓았다. 추가적으로, RAG 기반의 파이프라인에 필요한 vectorstore, qa_chain, llm을 None값으로 설정하고 이 값들을 setup_rag_pipeline 메소드를 통해 설정해준다.

```

def setup_rag_pipeline(self):
    # OpenAI LLM 설정
    self.llm = ChatOpenAI(model="gpt-4o-mini", temperature=0, max_tokens=500, openai_api_key=self.openai_api_key)
    # vectorstore 생성
    self.setup_vectorstore()
    # RAG 체인 생성 (질문과 문서를 결합하여 답변을 생성하는 체인)
    self.qa_chain = self.create_rag_chain()

```

RAG 체인에 활용하기 위한 LLM을 설정한다. gpt-4o-mini 모델을 활용하였으며, chatbot.py에서 RAG를 구현한 것과 같이 RAG 답변의 일관성을 주기 위해 temperature를 0으로 설정하고, max_tokens를 넉넉히 500으로 설정한다.

setup_vectorstore 메소드를 활용해 vectorstore를 생성하고, create_rag_chain 메소드를 이용해 qa_chain을 생성한다. 앞서 언급한 두 개의 메소드는 PDFRAGChatbot에서 구현한 것과 동일한 로직으로 설계한다.

```

def generate_responses(self):
    """
    복수의 질문에 대한 복수의 답변을 생성하는 메서드.
    """
    for question in self.questions:
        # 질문마다 관련 문서 검색
        retrieved_docs = self.retriever.invoke(question)

        # 검색된 문서가 없으면 빈 결과 처리
        if not retrieved_docs:
            self.answers.append("검색된 문서가 없습니다.")
        else:
            # 검색된 문서를 기반으로 답변 생성
            response = self.qa_chain.invoke(question)

            self.contexts.append([doc.page_content for doc in retrieved_docs])
            self.answers.append(response)

```

EvalRAGBot에 설계된 generate_responses 메소드이다. questions(질문 리스트)에서 각 질문에 대한 관련 문서를 설정했던 retriever를 이용해 검색하고, 검색된 문서를 기반으로 질문에 대한 RAG 답변(response)을 생성한다. 검색한 문서에서 context를 뽑아내고 이를 앞서 정의한 contexts 리스트에 저장하고, RAG 답변도 동일하게 answers 리스트에 저장한다.


```

def evaluate_rag_performance(self):
    """
    RAG 평가 수행 메서드.
    """
    self.generate_responses()

    data_samples = {
        'question': self.questions,
        'answer': self.answers,
        'ground_truth': self.gts,
        'contexts': self.contexts
    }
    dataset = Dataset.from_dict(data_samples)
    print("::-----Saving RAG Evaluation Dataset-----:")
    # rag 평가 데이터셋 저장
    dataset.to_csv('./rag_eval_dataset.csv')

    print("::-----Evaluation Start-----:")
    # Run the evaluation using RunEvalConfig
    # RunConfig로 evaluate customizing
    # 병렬 처리 포기하고 좋은 평가 결과 도출에 집중
    results = evaluate(dataset,
                        raise_exceptions=True,
                        run_config=RunConfig(
                            timeout = 240,
                            max_retries = 10,
                            max_wait = 180,
                            max_workers = 1,
                            exception_types = (ValidationError),
                        ),
                        metrics=[faithfulness,
                                answer_relevancy,
                                context_precision,
                                context_recall])

    return results

```

앞서 설명한 generate_responses 메소드로 RAG 답변 리스트와 contexts 리스트를 생성하고, data_samples 변수에 딕셔너리 형태로 저장한다. 이를 Dataset을 생성하여 'rag_eval_dataset.csv'라는 CSV 파일로 저장한다.

RAGAS의 evaluate 함수를 이용해 앞서 정의한 dataset을 바탕으로 'faithfulness', 'answer_relevancy', 'context_precision', 'context_recall' 평가를 진행한다. 이 때, faithfulness 계산이 오래 걸리므로, evaluate를 수행하는데 필요한 config 값들을 정의하는 RunConfig를 이용해 timeout과 max_retries, max_wait을 큰 값으로 잡아준다. 추가적으로, context_recall 계산 중 간혹 발생하는 에러로 인해 평가 중단되는 상황을 방지하기 위해 ValidationError를 예외처리한다.


```

def main():
    # QA 데이터셋 불러오기 및 가공
    df = pd.read_excel('./data/pdf_목록.xlsx', engine="openpyxl")

    # Ground Truth를 로드
    ground_truth = load_ground_truth(df)

    # ETF 종목명에 맞게 질문과 답변을 생성
    questions, gts = prepare_questions_and_gts(ground_truth)

    base_dir_temp = './data/간이투자설명서/'
    base_dir_monthly = './data/월간운용보고서/'

    pdf_files_temp = [os.path.join(base_dir_temp, file) for file in os.listdir(base_dir_temp) if file.endswith('.pdf')]
    pdf_files_monthly = [os.path.join(base_dir_monthly, file) for file in os.listdir(base_dir_monthly) if file.endswith('.pdf')]

    pdf_files = pdf_files_temp + pdf_files_monthly

    eval_rag = EvalRAGBot(pdf_files,
                           questions,
                           gts,
                           OPENAI_API_KEY,
                           PRJ_DIR,
                           )
    result = eval_rag.evaluate_rag_performance()
    print(result)
    # pandas dataframe
    result_df=result.to_pandas()
    result_df.to_csv('./ragas_evaluation_result.csv')

if __name__ == "__main__":
    main()

```

실질적인 실행이 수행되는 main 함수이다. 'pdf_목록.xlsx' 파일을 이용해 Question-GroundTruth 데이터셋을 불러와 utils.py에서 설명한 load_ground_truth 함수와 prepare_questions_and_gts 함수를 이용하여 questions 리스트와 GroundTruths 리스트를 생성한다.

'간이투자설명서'와 '월간운용보고서' 폴더 각각에 저장된 PDF 파일 총 40개에 대한 경로를 리스트에 각각 저장하여 pdf_files 리스트에 결합하였고, EvalRAGBot 클래스의 인스턴스(eval_rag)를 생성하여 evaluate_rag_performance 메소드를 실행시켜 평가를 진행하도록 설계한다.

평가결과는 Terminal 창에 print되며, 각 답변에 대한 평가지표는 'ragas_evaluation_result.csv' 파일에 저장된다.

[결과 화면 캡처]

